

Deep Learning Concepts



Dr. Joydeep Chandra,
Associate Professor,
Dept. of Computer Science & Engineering
Indian Institute of Technology Patna
joydeep@iitp.ac.in
<https://www.iitp.ac.in/~joydeep>

Some slides are adapted from Dr. Arijit's CS551 lecture

What we learn today

- Introduction to learning problems
- Features and their representations
- Artificial Neural Networks and foundations of Deep Learning
- Introduction to Orange Data Mining Tool
- Model performance optimization
- Introduction to Tensor Flow
- Implementation of Deep Learning Foundation models

Books

- Deep Learning - Ian Goodfellow, Yoshua Bengio, Aaron Courville
- The Elements of Statistical Learning - Jerome H Friedman, Robert Tibshirani, Trevor Hastie
- Reinforcement Learning: An Introduction - Richard S Sutton, Andrew G Barto
- Neural Network and Learning Machines - Simon S. Haykin

Learning and its importance

Problem space

- **Problems** --- *a well-defined task to derive some output given some input data.*
- **Target is to solve the same on a computer** --- *Generate a model that learns a mapping from inputs to outputs based on data*

Problem space

- **Problems** --- *a well-defined task to derive some output given some input data.*
- **Target is to solve the same on a computer** --- *Generate a model that learns a mapping from inputs to outputs based on data*
- Problems can be **intellectually challenging** for human being but relatively **straight forward** for a computer
 - Travelling salesman problem, chess
- Problems can be **easy** for common people but **difficult** for computer (even expressing it in a formal way)
 - Identifying an object, car (say), in a picture

Problem space

- **Problems** --- *a well-defined task to derive some output given some input data.*
- **Target is to solve the same on a computer** --- *Generate a model that learns a mapping from inputs to outputs based on data*
- Problems can be **intellectually challenging** for human being but relatively **straight forward** for a computer
 - Travelling salesman problem, chess
- Problems can be **easy** for common people but **difficult** for computer (even expressing it in a formal way)
 - Identifying an object, car (say), in a picture
- Primary focus will be in the *second category problems*

Problem Solving Strategies for Big Data

- Need to **solve** problems efficiently and accurately when the input data is huge ($\sim$ GB, TB order)
- Finding a deterministic algorithm is **difficult**
 - Need to find out features
 - Requires significant effort for model building
 - Need to have domain knowledge
- **Statistical inference** is found to be suitable
 - Feature selection is not crucial
 - Model will learn from past data

Common Data Analysts Use Cases

Descriptive and Diagnostic Analytics

Business Performance and KPI Tracking

- Sales performance dashboards
- Revenue growth analysis
- Conversion rate tracking
- Customer acquisition cost (CAC) analysis

Industries: Retail, SaaS, E-commerce, Finance

Customer Behavior Analysis

- Customer segmentation
- Churn analysis
- Funnel analysis (drop-off points)
- Cohort analysis

Industries: Telecom, Banking, E-commerce

Marketing and Campaign Analytics

- A/B testing analysis: Compare 2 versions; which is better?
- Campaign ROI measurement: How profitable a campaign is?
- Attribution analysis: Which channels or actions actually contributed to a positive outcome?
- Click-through and conversion analysis: How users respond to content and how many of them complete an action

Industries: Digital marketing, Media, Retail

Financial and Risk Analysis

- Profit and loss (P&L) analysis
- Budget forecasting
- Expense optimization
- Credit risk and default trends

Industries: Banking, Insurance, FinTech

Operation and Supply Chain Analytics

- Inventory optimization
- Demand forecasting
- Supplier performance analysis
- Process bottleneck identification

Industries: Manufacturing, Logistics

Data Quality and Auditing

- Missing value analysis
- Data consistency checks
- Anomaly detection in reports
- Master data validation

Industries: All data-driven organizations

Use of Machine Learning for Data Analysts

Use statistical models for:

- Regression Analysis
- Classification
- Clustering
- Time-series trend analysis

Data Analysts typically won't deal with

- Advanced ML deployments or model development
- Principle role of Data Scientists and AI Engineers

This part of the course attempts to cover both the roles

Learning Applications

Learning predictive outcomes

- Customer churn prediction (Who is likely to leave?)
- Customer lifetime value estimation (How much a customers is worth to the business)
- Demand & sales forecasting
- Credit risk & fraud detection
- User engagement & retention prediction
- HR attrition prediction

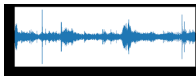
Optimization and Intelligence

- Recommendation systems (products, content, offers)
- Marketing response & uplift modeling
- Dynamic pricing & revenue optimization
- Supply chain optimization & predictive maintenance
- Personalization systems
- Text, image, and multimodal intelligence

Learning Goal

Data

	age (n)	job (c)	marital (c)	education (c)	balance (n)	housing (c)
0	30	unemployed	married	primary	1787	no
1	33	services	married	secondary	4789	yes
2	35	management	single	tertiary	1350	yes
3	30	management	married	tertiary	1476	yes
4	59	blue-collar	married	secondary	0	yes
5	35	management	single	tertiary	747	no



"Subject: naturally irresistible your corporate identity. It is really hard to recall
at a company: the market is full of suggestions and the information is overwhelming;
but a good catchy logo, stylish stationery and outstanding website will make the task
much easier. we do not promise that having ordered a logo your company will auto-
matically become a world leader. It is quite clear that without good products, effec-
tive business organization and practicable site it will be hot air nowadays market; but
we do promise that your marketing efforts will become much more effective. Here is t
he list of clear benefits: creativeness: hand-made, original logos, specially d
one to reflect your distinctive company image; convenience: logo and stationery ar
e provided in all formats; easy-to-use content management system lets you change
your website content and even its structure; promptness: you will see logo drafts w
ithin three business days; affordability: your marketing break-through should n
t make gaps in your budget; 100% satisfaction guaranteed: we provide unlimited am
ount of changes with no extra fees for you to be sure that you will love the result of
this collaboration, have a look at our portfolio

Tasks

Prediction

- Chances of being (or not) a defaulter
- Road condition (Smooth, bumpy)
- Genre (Pop, Jazz)
- Sentiment (Positive/Negative/Neutral)

Group

- Which set of users have similar credit behavior?
- Which roads have similar conditions?
- Which songs of similar genre
- Which reviews discuss similar aspects?

Introduction to Orange Data Mining Tool

An Open Source No-Code Solution to ML Tasks

Orange Data Mining Tool

Developed at University of Ljubljana, Slovenia

Download from <https://orangedatamining.com/>

Follow the instructions being given in class

Tutorial videos can be found at:

<https://www.youtube.com/playlist?list=PLmNPvQr9Tf-ZSDLwOzxpY-HrE0yv-8Fy>

Alternatives of Orange

- Google Cloud AutoML/Vertex AI
- Microsoft Azure AutoML
- IBM Watson AutoAI

Features and their representations

What are feature representations?

- The way we can describe data so that a machine learning model can use it.
- A feature is an individual measurable property describing the data.
 - Height, weight → features (numeric)
 - Categorical label (red, blue, green) → one-hot features
 - Image pixel intensity → features
 - Word embedding vector → features
 - Graph node degree → feature
- Raw data, like text, images, audio, logs, graphs, is usually not directly usable by ML models.
- We must convert it into numeric, meaningful, and compact representations.

Issue of Representation

- Representation of data in an efficient/structured manner is **crucial** for solving problems more effectively
 - Searching of a set of elements in a given list (sorted/unsorted)
 - Arithmetic operations on Arabic and Roman numerals
 - Primality test of n when n is represented as $11111 \dots 111$ (n -number of one)
- **Structured representation** can help in predicting future values

Learning representation/feature

- Traditional approaches
 - Pattern recognition
 - Input, output of the problem
- End to end learning
 - System automatically learns internal representation

AI-ML Tasks

- Heavily depends on **features**
- Requires **good** domain knowledge
- Feature extraction is **not** easy job
 - **Identify a car**
 - How to describe wheel
 - Shadow/brightness
 - Obscuring element

Representation Learning

- Learned representation often result in **better** performance compared to hand design
- Allows the system to rapidly **adapt** to new task
- Need to discover a good set of **features**
- Manual design of features is nearly **impossible**

Design of Features

- Goal is to separate out **variation factors**
- These factors are separate **sources of influence**
- It may exist as unobserved object or unobserved forces that **affect observable quantity**
 - Speech - Factors are age, sex, accent, etc
 - Image - Position, color, brightness, etc.

Deep Learning Introduction

Applications: Computer vision

- 2d to 3d conversion
- Street view generation
- Image classifications
- Image segmentation



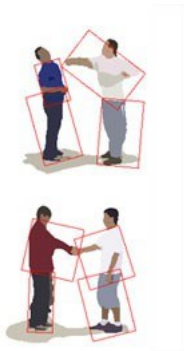
2D



3D

Applications: Activity Recognition

- Recognize activities like walking, running, cooking, etc. from still image or video data



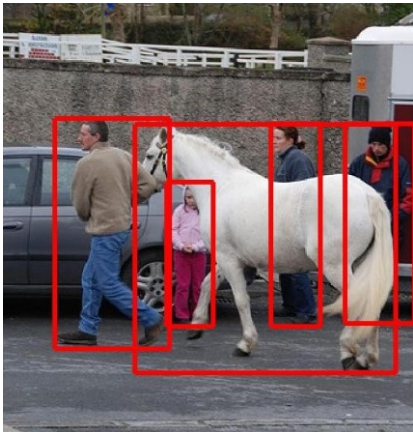
Applications: Image Captioning

- Automated caption generation for a given image

Describes without errors	Describes with minor errors	Somewhat related to the image	Unrelated to the image
 A person riding a motorcycle on a dirt road.	 Two dogs play in the grass.	 A skateboarder does a trick on a ramp.	 A dog is jumping to catch a frisbee.
 A group of young people playing a game of frisbee.	 Two hockey players are fighting over the puck.	 A little girl in a pink hat is blowing bubbles.	 A refrigerator filled with lots of food and drinks.
 A herd of elephants walking across a dry grass field.	 A close up of a cat laying on a couch.	 A red motorcycle parked on the side of the road.	 A yellow school bus parked in a parking lot.

Applications: Object Identification

- Identify objects in still image or in video stream



Applications: Automated Car

- Self driving car



Image source: Internet

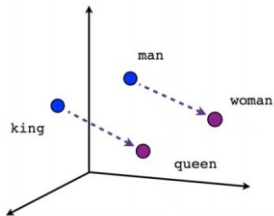
Applications: Drones & Robots

- Managing movement of robot or drones

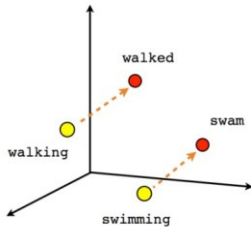


Applications: Natural Language Processing

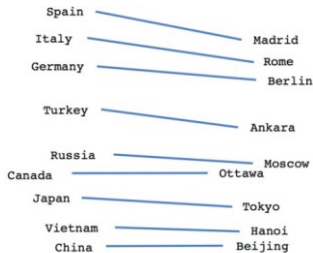
- Recommender system
- Sentiment analysis
- Question answering
- Information extraction from website
- Automated email reply



Male-Female



Verb tense



Country-Capital

Applications: Speech processing

- **Conversion of speech into text**
- **Generation of particular voice for a given text**



Image source: Internet

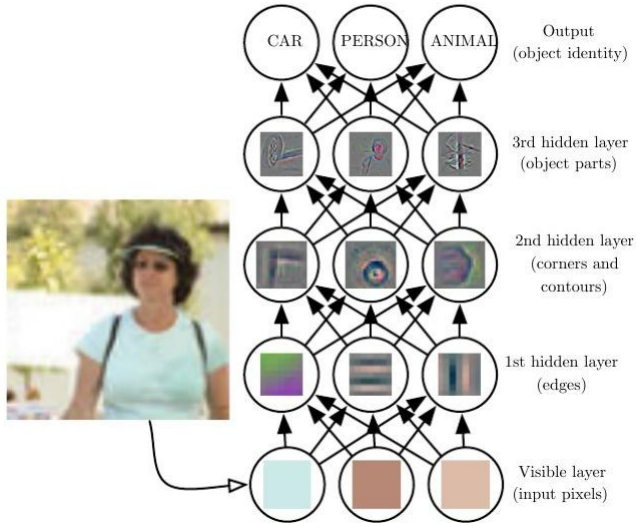
Other possible applications

- Language translation
- Weather prediction
- Genomics
- Drug discovery
- Particle physics
- Surveillance
- Cryptography and many more.

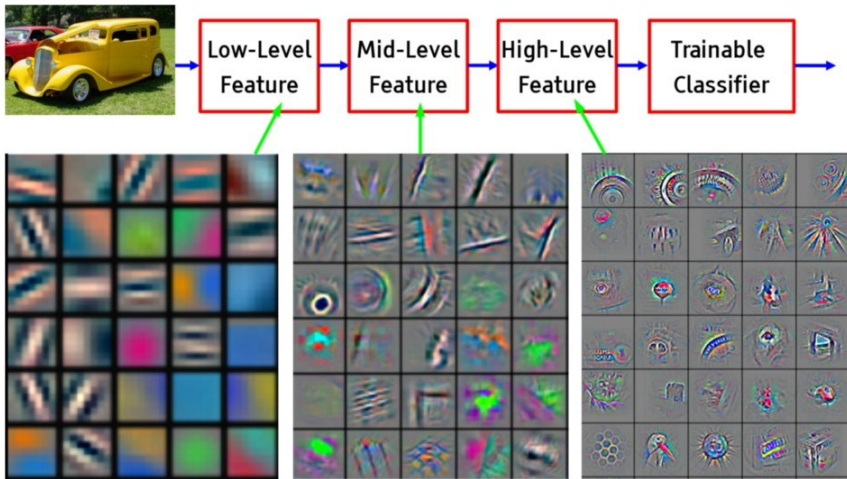
Deep Learning

- Try to address the problem of **representation learning**
- Representation are **expressed** in terms of other simpler representation
- Develop **complex concept** using simpler concept

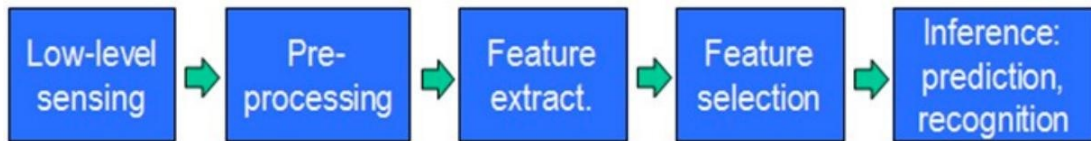
Simple to Complex Features



Simple to Complex Features



Conventional Machine Learning

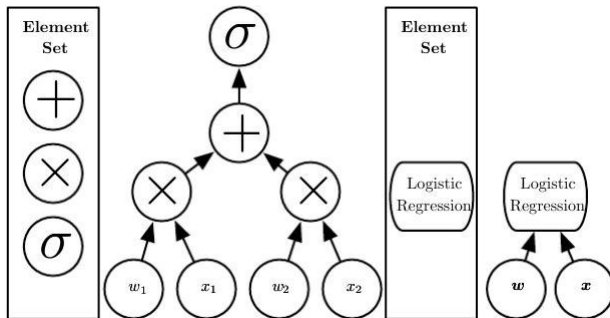


Deep Learning Model

- Feed-forward deep network or multilayer perceptron
- Mathematical functions that map input to output
- Composed of simpler functions
- Each layer provides a new representation
- Learning right representation

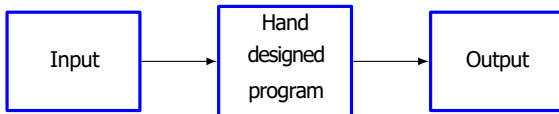
Depth of network

- Number of **sequential instruction** must be executed to evaluate the architecture
 - Length of the longest path
- Depth of the model

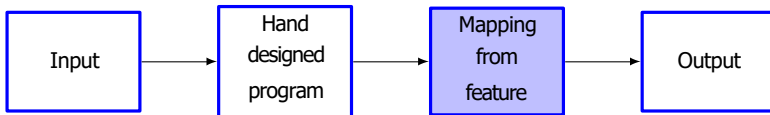


Representation learning

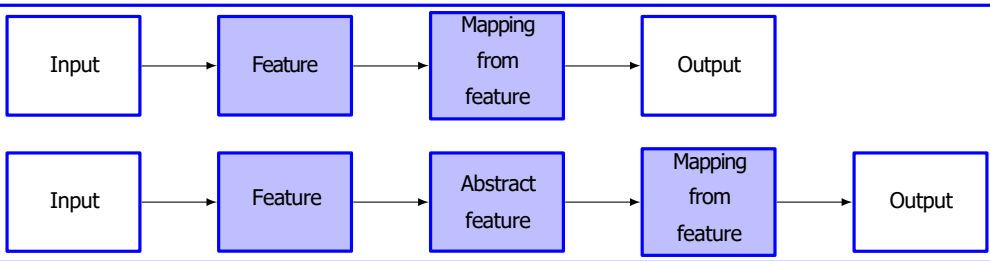
Rule based
system



Classic
machine
learning



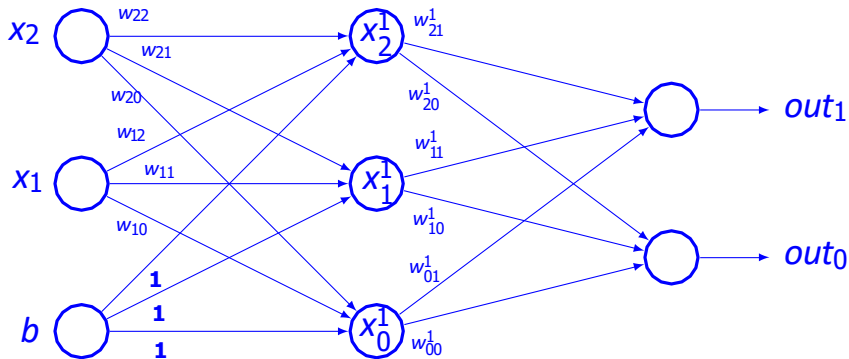
Deep
Learning



Artificial Neural Networks – Foundation of Deep Learning

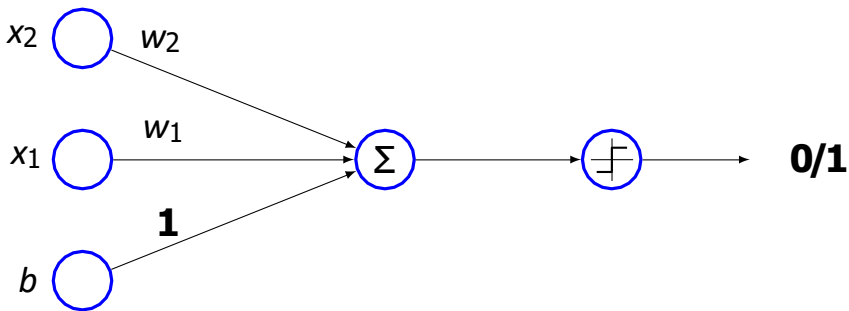
Artificial Neural Network

- A simple model



Neuron

- One of the primitive models



Artificial Neuron as classifier

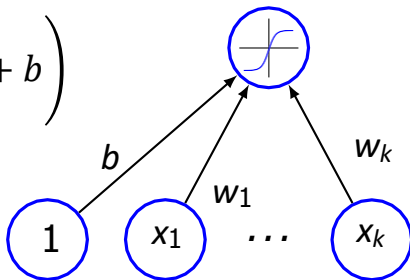
- Neuron pre-activation function

$$a(x) = \sum_i w_i x_i + b = b + w^T x$$

- Neuron output activation function

$$h(x) = g(a(x)) = g\left(\sum_i w_i x_i + b\right)$$

- b — Neuron bias
- $g(\cdot)$ — Activation function



Learning weights and biases

Given the following data:

x_1	x_2	Output $o(X)$
0	0	0
0	1	1
1	0	1
1	1	1

- How to find the weights and bias?
- Derive the weights and bias such that the predicted output matches the actual output
- Loss function

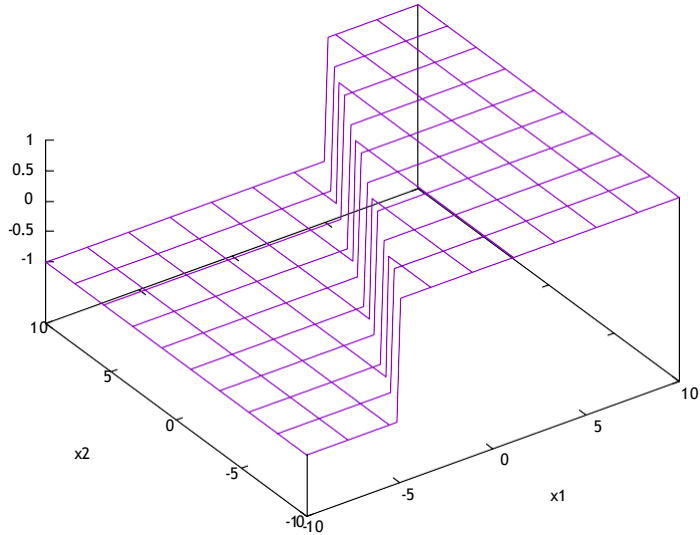
$$\begin{aligned}\mathcal{L} &= \sum_i \left(h(X^{(i)}) - o(X^{(i)}) \right)^2 \\ &= \sum_i \left(g(w_1 x_1^{(i)} + w_2 x_2^{(i)} + b) - o(X^{(i)}) \right)^2\end{aligned}$$

- Optimize $\mathcal{L}$ using stochastic gradient descent

Learning weights and biases (stochastic gradient)

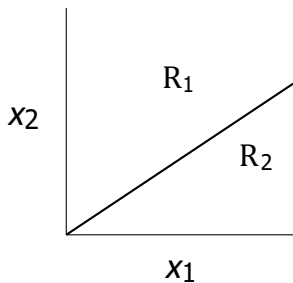
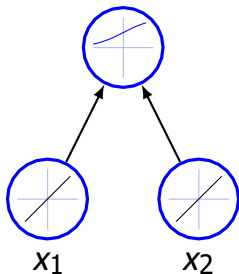
- Start with some random weight values of w_1 , w_2 and b
- Run the model with these weights for a data instance (or a batch of data instances)
 - Calculate the loss $\mathcal{L}$
- Compute the gradients $\frac{\partial \mathcal{L}}{\partial w_1}$, $\frac{\partial \mathcal{L}}{\partial w_2}$ and $\frac{\partial \mathcal{L}}{\partial b}$
- Update the weights and bias as
 - $w_1^{new} = w_1^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_1}$
 - $w_2^{new} = w_2^{old} - \eta \frac{\partial \mathcal{L}}{\partial w_2}$
 - $b^{new} = b^{old} - \eta \frac{\partial \mathcal{L}}{\partial b}$
- Repeat the above steps several times (epochs)

Physical interpretation



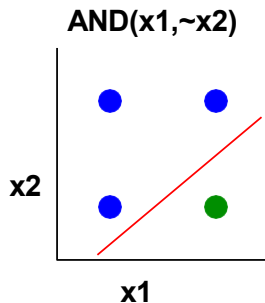
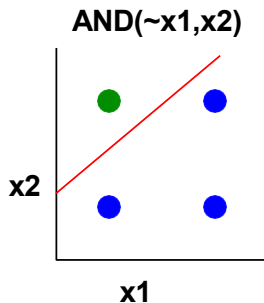
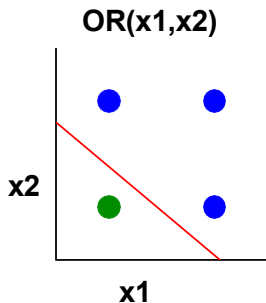
Classification using single neuron

- **Single neuron can do binary classification**
 - Also known as logistic regression classifier



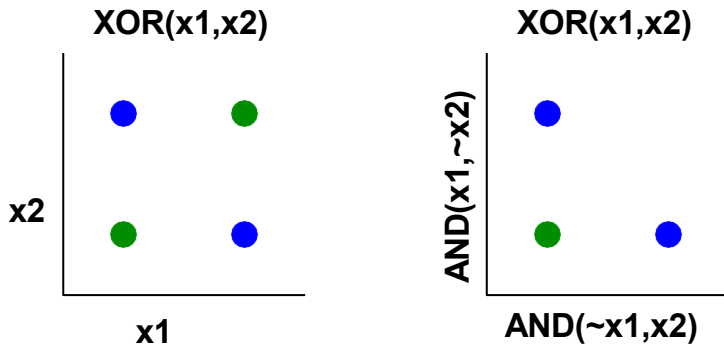
Artificial neuron

- Can solve linearly separable problems



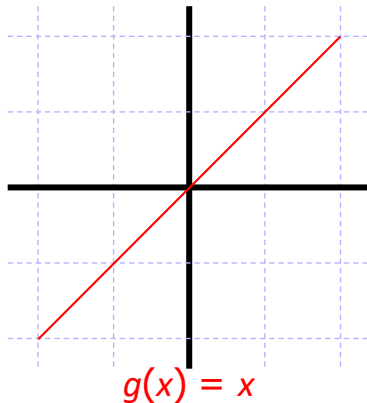
Artificial neuron: XOR problem

- There are issues for linear separation



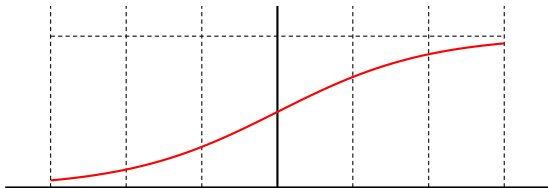
Activation function

- **Linear activation function**
 - Not very interesting
 - No change in values
 - Huge range



Activation function

- Sigmoid function
 - Values lie between 0 and 1
 - Strictly increasing function
 - Bounded

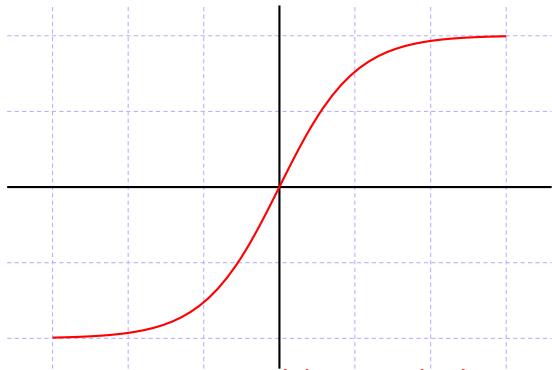


$$g(x) = \text{sigm}(x) = \frac{1}{1 + \exp(-x)}$$

Activation function

- **Hyperbolic Tangent (Tanh) function**

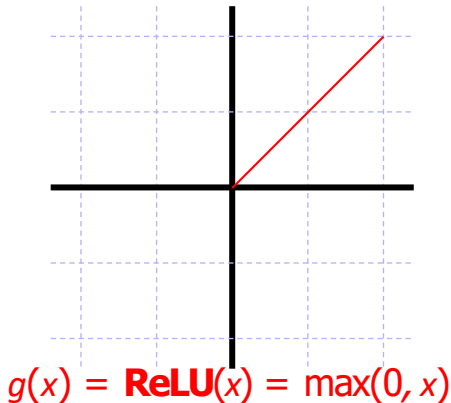
- Can be positive or negative
- Values lie between -1 and 1
- Strictly increasing function
- Bounded



$$g(x) = \tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

Activation function

- Rectified linear activation function
 - Bounded below by 0
 - Strictly increasing function
 - Not upper bounded



Single hidden layer neural network

- Hidden layer pre-activation

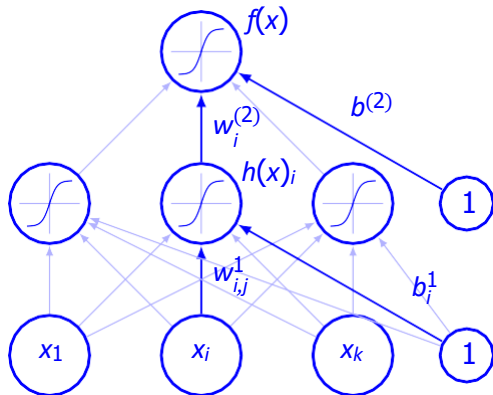
$$a(\mathbf{X}) = \mathbf{b}^{(1)} + \mathbf{W}^{(1)}\mathbf{X}$$

- Hidden layer activation

$$h(\mathbf{X}) = g(a(\mathbf{X}))$$

- Output layer activation

$$f(\mathbf{X}) = \hat{o}(b^{(2)} + \mathbf{w}^{(2)T}h^{(1)}(\mathbf{X}))$$



Deep feedforward networks

- Also known as feedforward neural network or multilayer perceptron
- Goal of such network is to approximate some function f^*
 - For classifier, x is mapped to category y i.e. $y = f^*(x)$
 - A feedforward network maps $y = f(x; v)$ and learns v for which the result is the best function approximation
- Information flows from input to intermediate to output
 - No feedback, directed acyclic graph
 - For general model, it can have feedback and known as recurrent neural network
- Typically it represents composition of functions
 - Three functions $f^{(1)}, f^{(2)}, f^{(3)}$ are connected in chain
 - Overall function realized is $f(x) = f^{(3)} \left(f^{(2)} \left(f^{(1)}(x) \right) \right)$
 - The number of layers provides the depth of the model

Multi layer neural network

- Pre-activation in layer

$$k > 0 \quad (h^{(0)}(\mathbf{X}) = \mathbf{X})$$

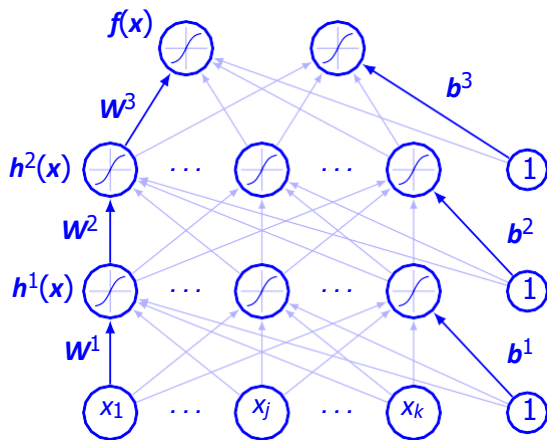
$$a^{(k)}(\mathbf{X}) = b^{(k)} + \mathbf{W}^{(k)} h^{(k-1)}(\mathbf{X})$$

- Hidden layer activation

$$h^{(k)}(\mathbf{X}) = g(a^{(k)}(\mathbf{X}))$$

- Output layer activation

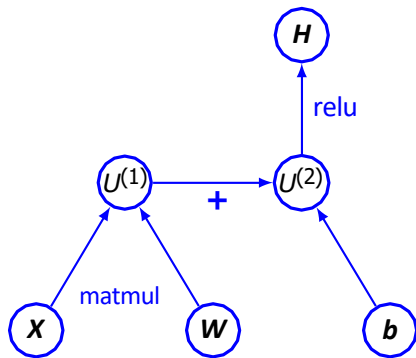
$$f(\mathbf{X}) = h^{(L+1)}(\mathbf{X}) = \hat{o}(a^{(L+1)}(\mathbf{X}))$$



Back propagation

- In a feedforward network, an input x is read and produces an output $\hat{y}$
 - This is forward propagation
- During training forward propagation continues until it produces cost $\mathcal{L}(\theta)$
- Back-propagation algorithm allows the information to flow backward in the network to compute the gradient
- Computation of analytical expression for gradient is easy
- We need to find out gradient of the cost function with respect to the parameters i.e., $\nabla_{\theta} \mathcal{L}(\theta)$

Computational graph



Chain rule of calculus

- Back-propagation algorithm heavily depends on it
- Let x be a real number and $y = g(x)$ and $z = f(g(x)) = f(y)$
- Chain rule says $\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$

Computational graph & subexpression

- We have $x = f(w)$, $y = f(x)$, $z = f(y)$

$$\frac{\partial z}{\partial w}$$

$$= \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \frac{\partial x}{\partial w}$$

$$= f'(y)f'(x)f'(w)$$

$$= f'(f(f(w)))f'(f(w))f'(w)$$



Forward propagation in MLP

- Input

$$h^{(0)} = x$$

- Computation for each layer $k = 1, 2, \dots, l$

$$a^{(k)} = b^{(k)} + W^{(k)}h^{(k)}$$

$$h^{(k)} = f(a^{(k)})$$

- Computation of output and loss function

$$\hat{y} = h^{(l)}$$

$$J = \mathcal{L}(\hat{y}, y) + \lambda \Omega(\theta)$$

Backward computation in MLP

- Compute gradient at the output

$$g \leftarrow \nabla_{\hat{y}} \mathcal{L} = \nabla_{\hat{y}} \mathcal{L}(\hat{y}, y)$$

- Convert the gradient at output layer into gradient of pre-activation

$$g \leftarrow \nabla_{a^{(k)}} \mathcal{L} = g \odot f'(a^{(k)})$$

- Compute gradient on weights and biases

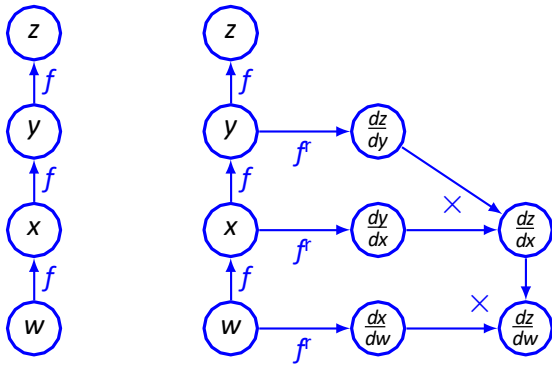
$$\nabla_{b^{(k)}} \mathcal{L} = g + \lambda \nabla_{b^{(k)}} \Omega(\theta)$$

$$\nabla_{W^{(k)}} \mathcal{L} = g h^{(k-1)T} + \lambda \nabla_{W^{(k)}} \Omega(\theta)$$

- Propagate the gradients wrt the next lower level activation

$$g \leftarrow \nabla_{h^{(k-1)}} \mathcal{L} = W^{(k)T} g$$

Example



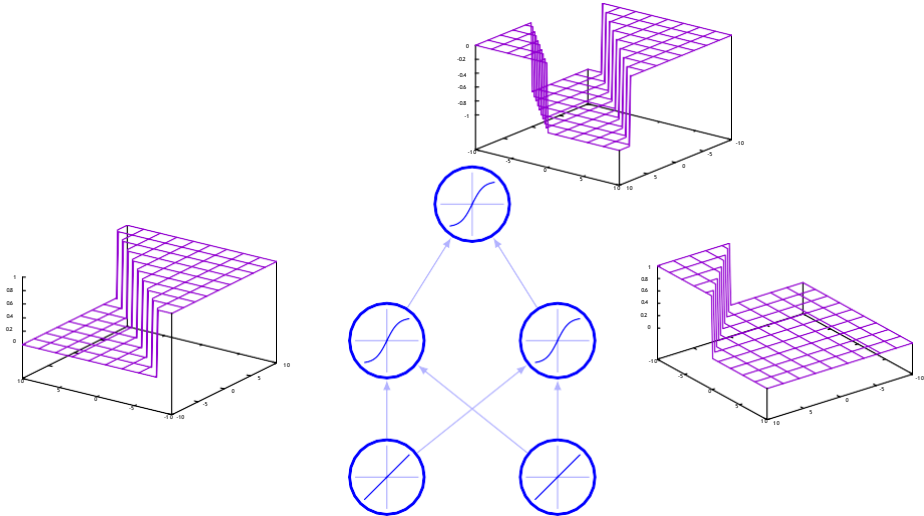
Multiclass classification

- Need multiple outputs that is one neuron for each class
- Need to determine probability of $p(y = c | x)$
- Softmax activation function is used at the output

- $$o(a) = softmax(a) = \left[\frac{\exp(a_1)}{\sum_c \exp(a_c)}, \frac{\exp(a_2)}{\sum_c \exp(a_c)}, \dots, \frac{\exp(a_n)}{\sum_c \exp(a_c)} \right]^T$$

- Strictly positive
- Sum to 1
- Class having the highest probability will be the predicted output

Capacity of neural network



Capacity of neural network

- Universal approximation theorem (UAT) (Hornik,1991)
 - A single hidden layer neural network with a linear output unit can approximate any continuous function arbitrarily well, given enough hidden units.
- The result is applicable for other hidden layer activation functions such as sigmoid, tanh, etc.
- This is a promising result, but it does not say that there is a learning algorithm to find the necessary parameter values!

If UAT holds why deep learning is required?

UAT provides theoretical guarantee but not practicality. Deep Learning provide

- Efficiency
 - Shallow networks may need exponentially many neurons.
 - Deep networks need far fewer neurons → compact & scalable.
- Hierarchical Feature Learning
 - Images: (edges → shapes → objects) Text: (characters → words → sentences)
 - Deep layers naturally learn multi-level representations.
- Better Optimization
 - Depth enables gradient flow, residual connections, normalization.
 - Shallow “wide” networks are hard to train.
- Better Generalization
 - Deep models impose useful structure (e.g., convolution, attention).
 - Shallow networks often overfit or learn unstable functions.
- Compositionality
 - Many real-world tasks are compositions of simpler functions.
 - Deep networks represent these efficiently; shallow ones cannot.

Model Optimization and Performance Improvement

Model Optimization: Overview

- Goal: Improve model accuracy, efficiency, and generalization
- Optimization applies to training, architecture, and deployment
- Trade-offs: performance vs interpretability vs compute

Data-Centric Optimization

- Data cleaning and noise removal
- Data augmentation
- Balanced and representative datasets

Data Cleaning of Tabular Data

- Handling missing values
 - Mean / median / mode imputation
 - KNN imputation
 - Model-based imputation
- Duplicate removal
- Outlier detection and treatment
 - Z-score, Inter Quartile Range (IQR)
 - Methods like Isolation Forest
- Data type correction
 - Numeric vs categorical fixes
- Inconsistent value correction
 - Standardizing units (e.g., kg vs lbs)
 - Category normalization (Yes / Y / yes)

Noise removal from Tabular Data

- Measurement errors
 - Sensor reading = 10.3 instead of 1.03
- Data entry errors
 - Age = 230 instead of 23
- Inconsistent units
 - Salary in thousands mixed with full values
- Random fluctuations in numeric fields

Techniques adopted

- Smoothing using binning
- Robust scaling $x_{scaled} = \frac{x - median}{Inter\ Quartile\ Range}$
- Winsorization (clipping extreme values)
- Noise-aware models (Huber loss)

Data cleaning and noise removal in unstructured text data

Data Cleaning Techniques

- Lower casing
- Removing punctuation and special characters
- Stop-word removal
- Spell correction
- Removing URLs, emails, emojis
- De-duplication of documents
- Language detection & filtering

Noise Removal

- Token normalization (stemming, lemmatization)
- Removing rare or very frequent tokens
- Character-level noise filtering
- OCR noise correction (for scanned text)
- Sentence filtering (very short or very long sentences)

Data cleaning and noise removal in image data

Data Cleaning Techniques

- Removing corrupted or unreadable images
- Removing duplicate images
- Standardizing image size and format
- Color normalization
- Label correction (manual or semi-automated)

Noise Removal

- Image denoising filters like Gaussian, Median and Bilateral filters
- Background removal
- Contrast normalization
- Blur and artifact correction
- Data augmentation to reduce noise sensitivity

Feature Engineering & Selection (If using traditional ML)

- Feature scaling and normalization
- Encoding categorical variables
- Feature selection (filter, wrapper, embedded methods)
- Dimensionality reduction (PCA, autoencoders)

Model Selection

- Choosing the right algorithm for the problem
 - Bias–variance tradeoff
 - Baseline models vs complex models
 - Ensemble vs single models

Hyperparameter Tuning

- Grid search
- Random search
- Bayesian optimization
- Automated hyperparameter tuning (AutoML)

Regularization Techniques

- L1 and L2 regularization
 - L1 regularization: Add $\lambda \sum_i |w_i|$ to the loss function to optimize.
 - Induces weight and feature sparsity
 - L2 regularization: Add $\lambda \sum_i w_i^2$ to the loss function to optimize
 - Produces small weights
 - Elastic net regularization: Combine L1 and L2 as $\lambda(\alpha \sum_i |w_i| + \beta \sum_i w_i^2)$
 - Here λ, α, β are hyperparameters to control importance of the regularization terms
- Dropout (for neural networks)
 - Set some of the weights to 0
- Early stopping
- Data-driven regularization

Training Optimization

- Learning rate scheduling
- Batch size tuning
- Optimizer choice (SGD, Adam, RMSprop)
- Proper weight initialization

Architecture Optimization (Deep Learning)

- Depth and width of networks
- Activation function selection
- Residual connections
- Model pruning and compression

Evaluation & Validation

- Cross-validation
- Appropriate evaluation metrics
- Error analysis
- Handling overfitting and underfitting

Example Use case

Process Company

Working with clients

→ Process delays

→ SLA breaches

→ Studies the client profile

→ Organization set

→ Turnover

→ # of employees

→ Headquarters

1) Preprocess the data

→ Clean data

→ Remove noise

→ Feature selection

↳ PCA

→ Filter & Wrapper techniques

→ Split into Train, Validate & Test

→ select models → classification

→ Regression

Logistic Regression

→ Different Scales of data

→ Categorical data

→ One hot encoded data

→ Regularization → L1
→ L2

Predict

→ ElasticNet

1) Whether there will be a SLA breach? (Classification)

OR

2) Amount of delay it may happen (Regression)

Logistic Regression
Decision Trees, XGBoost

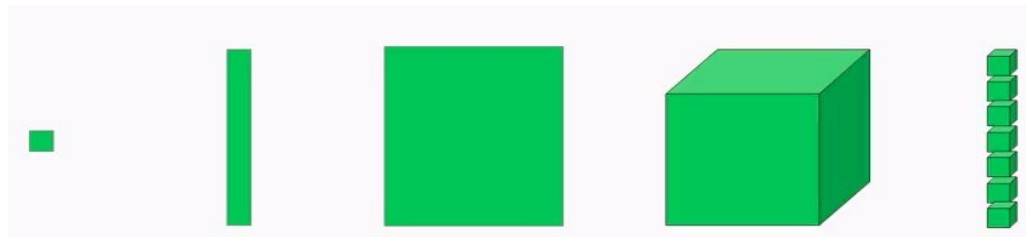
Building Multilayer Deep Neural Networks: TensorFlow and Keras

What are Tensors?

According to the Wikipedia:

A **tensor** is an algebraic object that describes a multilinear relationship between sets of algebraic objects related to a vector space.

In short, tensors are generalization of scalars and vectors



0d Tensor

1d Tensor

2d Tensor

3d Tensor

4d Tensor

Tensors Rank and Shape

(10)

Scalar

Rank = 0 Shape = (0)

Tensors Rank and Shape

(10)

Scalar

Rank = 0 Shape = (0)

1
2
3

Vector

Rank = 1 Shape = (3)

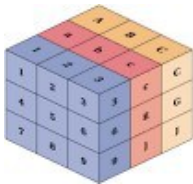
Tensors Rank and Shape



1	2
4	5
7	8

Matrix

Rank = 2 Shape = (3x2) 3 number of Rank 1 tensors of shape 2



Rank = 3 Shape = (3x3x3) 3 number of Rank 2 tensors of shape 3x3

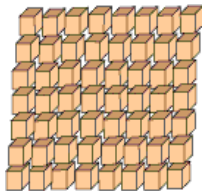
Tensors Rank and Shape



4 d -Tensor

Rank = 4 Shape = $(7 \times 3 \times 3 \times 3)$

7 number of Rank 3 tensors of shape $3 \times 3 \times 3$



5 d -Tensor

Rank = 5 Shape = $(7 \times 8 \times 3 \times 3 \times 3)$

7 number of Rank 4 tensors of shape $8 \times 3 \times 3 \times 3$

Tensors in TensorFlow

```
import tensorflow as tf  
  
t = tf.constant( 4 )  
print( t )
```



```
tf.Tensor( 4,      shape=(),      dtype=int32 )
```

Tensors in TensorFlow

```
import tensorflow as tf
```

```
t = tf.constant( 4 )
```

```
print( t )
```



```
tf.Tensor( 4, shape=(), dtype=int32 )
```

```
import tensorflow as tf
```

```
t = tf.constant( [2.0, 3.0, 4.0] )
```

```
print(t)
```



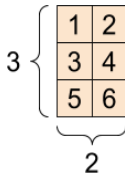
```
tf.Tensor([2. 3. 4.], shape=(3,), dtype=float32)
```

Tensors in TensorFlow

```
import tensorflow as tf
```

```
t = tf.constant( [[1, 2],  
                  [3, 4],  
                  [5, 6]  
                ])  
print(t)
```

```
tf.Tensor( [[1 2] [3 4] [5 6]], shape=(3, 2),  
dtype=float16)
```



No. of rows = 3

No. of Columns = 2

Tensors in TensorFlow

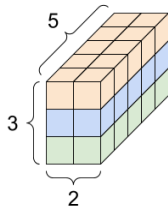
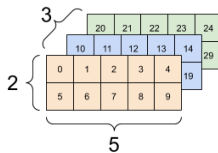
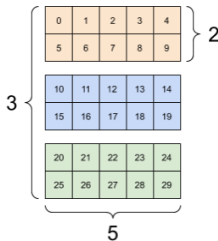
```
import tensorflow as tf
```

```
t = tf.constant([  
    [ 0, 1, 2, 3, 4],  
    [ 5, 6, 7, 8, 9] ],  
    [ [10, 11, 12, 13, 14],  
      [15, 16, 17, 18, 19] ],  
    [ [20, 21, 22, 23, 24],  
      [25, 26, 27, 28, 29] ]  
    ])
```

```
print(t)
```



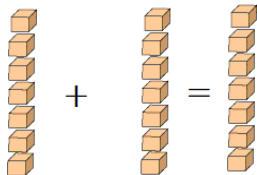
```
tf.Tensor( [[[ 0  1  2  3  4] [ 5  6  7  8  9]] [[10 11 12 13  
14] [15 16 17 18 19]] [[20 21 22 23 24] [25 26 27  
28 29]]], shape=(3, 2, 5), dtype=int32)
```



Operations between two Tensors

Given two tensors x and y ,

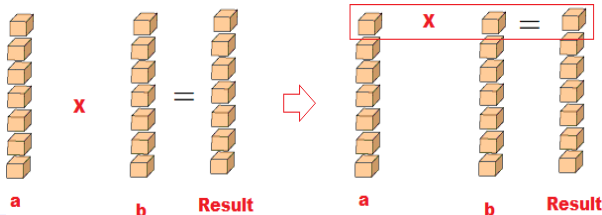
- Arithmetic operations such as **plus, minus, multiplication, division** can be performed between x and y , and produce another tensor.
 - $x + y$, $x - y$, $x * y$, x / y
- To perform a binary operation between two tensors, the shape of the two should be compatible.
- Element wise operations between the two tensors are performed.



Operations between two Tensors

Given two tensors x and y ,

- Arithmetic operations such as **plus, minus, multiplication, division** can be performed between x and y , and produce another tensor.
 - $x + y$, $x - y$, $x * y$, x / y
- To perform a binary operation between two tensors, the shape of the two should be compatible.
- Element wise operations between the two tensors are performed.



Broadcast (Stretch)

X (1d array): 3

Y (1d array 1

Result (1d array) : 3

Broadcast (Stretch)

X (1d array): 3 [1, 2, 3]

Y (1d array) 1 2

Result (1d array) : 3

Broadcast (Stretch)

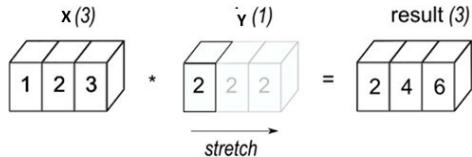
X (1d array): 3 [1, 2, 3]

Y (1d array) 1 2

Result (1d array) : 3

TensorFlow performs broadcast of the lower shape tensor.

It means, the low dimensional tensor is replicated till we find the matching shape



Broadcast (Stretch)

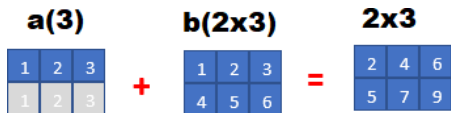
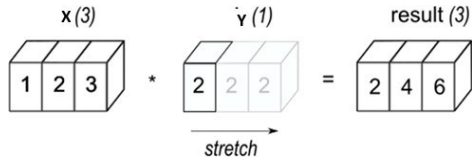
X (1d array): 3 [1, 2, 3]

Y (1d array) 1 2

Result (1d array) : 3

TensorFlow performs broadcast of the lower shape tensor.

It means, the low dimensional tensor is replicated till we find the matching shape



Two Sides Broadcast (Stretch)

X (1d array): 3

Y (2d array): 3 x 1

Result (2d array): 3 x 3

1	2	3
1	2	3
1	2	3

+

4	4	4
5	5	5
6	6	6

=

5	6	7
6	7	8
7	8	9

Compatibility of two Tensors for Arithmetic Operation

Two tensors **x** and **y** are said to be compatible if

Compatibility of two Tensors for Arithmetic Operation

Two tensors x and y are said to be compatible if

- Their dimensions and shapes are same
 - $\text{Dimension}(x) = \text{Dimension}(y)$
 - $\text{Shape}(x) = \text{Shape}(y)$

1	2	3
1	2	3

+

1	2	3
4	5	6

=

2	4	6
5	7	9

X (2d array): 2 x 4

Y (2d array): 2 x 4

Result (2d array) : 2 x 4

Compatibility of two Tensors for Arithmetic Operation

Two tensors x and y are said to be compatible if

- Their dimensions and shapes are same
 - $\text{Dimension}(x) = \text{Dimension}(y)$
 - $\text{Shape}(x) = \text{Shape}(y)$
- Their **dimensions** or **shapes** are different, but the following condition is satisfied.
 - For all dimension position, one of component dimension has shape 1.

X (2d array): 2 x 4

Y (2d array): 2 x 4

Result (2d array) : 2 x 4

X (2d array): 2 x 1

Y (2d array): 2 x 4

Result (2d array) : 2 x 4

Compatibility of two Tensors for Arithmetic Operation

Two tensors x and y are said to be compatible if

- Their dimensions and shapes are same
 - $\text{Dimension}(x) = \text{Dimension}(y)$
 - $\text{Shape}(x) = \text{Shape}(y)$
- Their **dimensions** or **shapes** are different, but the following condition is satisfied.
 - For all dimension position, one of component dimension has shape 1.

X (2d array): 2 x 4

Y (2d array): 2 x 4

Result (2d array) : 2 x 4

X (2d array): 2 x 1

X (3d array): 2 x 1 x 3

Y (2d array): 2 x 4

Y (3d array): 2 x 4 x 1

Result (2d array) : 2 x 4

Result (3d array) : 2 x 4 x 3

Compatibility of two Tensors for Arithmetic Operation

Two tensors x and y are said to be compatible if

- Their dimensions and shapes are same
 - $\text{Dimension}(x) = \text{Dimension}(y)$
 - $\text{Shape}(x) = \text{Shape}(y)$
- Their **dimensions** or **shapes** are different, but the following condition is satisfied.
 - For all dimension position, one of component dimension has shape 1.

X (2d array): 2 x 4

Y (2d array): 2 x 4

Result (2d array): 2 x 4

X (2d array): 2 x 1

Y (2d array): 2 x 4

Result (2d array): 2 x 4

X (3d array): 2 x 1 x 3

Y (3d array): 2 x 4 x 1

Result (3d array): 2 x 4 x 3

X (3d array): 2 x 1 x 3

Y (3d array): 1 x 4 x 1

Result (3d array): 2 x 4 x 3

Compatibility of two Tensors for Arithmetic Operation

Two tensors **x** and **y** are said to be compatible if

- Their dimensions and shapes are same

X (2d array): 2 x 3

- Dimension(x) = Dimension(y)

Y (2d array): 2 x 4

- Shape(x) = Shape(y)

Result (2d array) : 2 x 4

- Their **dimensions** and/or **shapes** are different, but the following condition is satisfied.
 - For all dimension position, one of component dimension has shape 1.

A (4d array): 8 x 1 x 6 x 1

B (3d array): 7 x 1 x 5

Result (4d array): 8 x 7 x 6 x 5

The resultant dimension is

- Higher dimension
- Higher shape of the component dimensions

They are not compatible

A (1d array): 3

B (1d array): 4 # *trailing dimensions do not match*

A (2d array): 2 x 1

B (3d array): 8 x 4 x 3 # *second from last dimensions mismatched*

Concept of axis in tensorflow

- Axis tells you which dimension of a tensor an operation should act on.
 - Useful for slicing, reducing, reshaping, and gathering data.
- For a 2D tensor (matrix) of shape (3, 4):
 - Axis 0 → rows (the outer list)
 - Axis 1 → columns (the inner list)

```
[  
  [ a, b, c, d ], ← axis 1 (columns)  
  [ e, f, g, h ],  
  [ i, j, k, l ]  
]  
↑  
axis 0 (rows)
```

Tensor shape	Rank	Axis meaning
(N,)	1D	axis 0: elements
(N, M)	2D	axis 0: rows, axis 1: columns
(B, H, W, C)	4D (images)	axis 0 = batch, axis 1 = height, axis 2 = width, axis 3 = channels
(T, B, F)	3D (RNN)	axis 0 = time, axis 1 = batch, axis 2 = features

- Axis 0 → operate across rows
- Axis 1 → operate across columns
- Axis 2 → depth/channel dimension (in images)
- Axis -1 → the last dimension
- Axis -2 → the second last dimension

Variable Tensor

A variable tensor is created using ***tf.Variable()*** function.

Syntax: `tf.Variable(initial_value=None, trainable=None, validate_shape=True, caching_device=None, name=None, variable_def=None, dtype=None, import_scope=None, constraint=None, synchronization=tf.VariableSynchronization.AUTO, aggregation=tf.compat.v1.VariableAggregation.NONE, shape=None)`

- ***initial_value***: by default None. The initial value for the Variable is a Tensor, or a Python object convertible to a Tensor.
- ***trainable***: by default None. If True, GradientTapes will keep an eye on this variable's usage.
- ***validate_shape***: by default True. Allows the variable to be initialised with an unknown shape value if False. The shape of initial value must be known if True, which is the default.
- ***name***: by default None. The variable's optional name. Defaults to 'Variable' and is automatically uniquified.
- ***variable_def***: by default None.
- ***dtype***: by default None. If set, *initial_value* will be converted to the given type. If None, either the datatype will be kept (if *initial_value* is a Tensor), or *convert_to_tensor* will decide.
- ***shape***: by default None. if None the shape of *initial_value* will be used. if any shape is specified, the variable will be assigned with that particular shape.

Few examples of Variable Tensor

```
import tensorflow as tf
```

```
x = tf.Variable( [1, 2, 3, 4] )
```

```
x = tf.Variable( [1.2, 4.4, 5, 6] )
```

```
x = tf.Variable( ['a', 'b', 'c', 'd'] )
```

```
x = tf.Variable( [True, False] )
```

```
x = tf.Variable( [3 + 4j] )
```

```
import tensorflow as tf
```

```
x = tf.Variable([1,2,3,4])  
print(x)
```

```
<tf.Variable 'Variable:0' shape=(4,) dtype=int32, numpy=array([1, 2, 3, 4])>
```

```
x = tf.Variable([[1,2,3,4],[5,6,7,8]])  
print(x)
```

```
<tf.Variable 'Variable:0' shape=(2, 4) dtype=int32, numpy= array([[1, 2, 3, 4],  
                                                                    [5, 6, 7, 8]])>
```

Find the attributes of Tensor

```
x = tf.Variable([1,2,3,4])
print(x.name)

print(x.shape)

print(x.dtype)

print(x.numpy())
```

```
Variable:0
(4,)
<dtype: 'int32'>
[1 2 3 4]
```

```
x = tf.Variable([[1,2,3,4],[5,6,7,8]])
print(x.name)

print(x.shape)

print(x.dtype)

print(x.numpy())
```

```
Variable:0
(2, 4)
<dtype: 'int32'>
[[1 2 3 4]
 [5 6 7 8]]
```

Constant tensor can be converted to Variable tensor

```
x_con = tf.constant([1,2,3,4])  
  
x_var = tf.Variable(x_con)  
print(x_var)
```

```
<tf.Variable 'Variable:0' shape=(4,) dtype=int32, numpy=array([1, 2, 3, 4])>
```

Variable tensor can be converted to Constant tensor

```
x_var = tf.Variable([1,2,3,4])  
x_con = tf.constant(x_var)  
print(x_con)
```

```
tf.Tensor([1 2 3 4], shape=(4,), dtype=int32)
```

Include data type as parameter

```
x = tf.constant([1,2,3,4], dtype=tf.float32)  
print(x)
```

```
tf.Tensor([1. 2. 3. 4.], shape=(4,), dtype=float32)
```

Reshape the structure of tensor

```
x = tf.constant([1,2,3,4,5,6])  
print(x)
```

```
tf.Tensor([1 2 3 4 5 6], shape=(6,), dtype=int32)
```

```
[ 1, 2, 3, 4, 5, 6]
```

Reshape the structure of tensor

```
x = tf.constant([1,2,3,4,5,6])  
print(x)
```

```
tf.Tensor([1 2 3 4 5 6], shape=(6,), dtype=int32)
```

```
[ 1, 2, 3, 4, 5, 6]
```

```
x = tf.constant([1,2,3,4,5,6], shape=(2,3))  
print(x)
```

```
tf.Tensor(  
  [[1 2 3]  
   [4 5 6]], shape=(2, 3), dtype=int32)
```

```
[ 1, 2, 3]  
[ 4, 5, 6]
```

Reshape the structure of tensor

```
x = tf.constant([1,2,3,4,5,6])  
print(x)
```

```
tf.Tensor([1 2 3 4 5 6], shape=(6,), dtype=int32)
```

```
[ 1, 2, 3, 4, 5, 6]
```

```
x = tf.constant([1,2,3,4,5,6], shape=(2,3))  
print(x)
```

```
tf.Tensor(  
[[1 2 3]  
 [4 5 6]], shape=(2, 3), dtype=int32)
```

```
[ 1, 2, 3]  
[ 4, 5, 6]
```

```
tf.reshape(x, (3,2))
```

```
<tf.Tensor: shape=(3, 2), dtype=int32, numpy=  
array([[1, 2],  
       [3, 4],  
       [5, 6]])>
```

```
[ 1, 2]  
[ 3, 4]  
[ 5, 6]
```

Flatten the tensor

[1, 2, 3, 4]
[5, 6, 7, 8]



[1, 2, 3, 4, 5, 6, 7, 8]

```
x = tf.constant([[1,2,3,4],[5,6,7,8]], shape=(8))  
print(x)
```

```
tf.Tensor([1 2 3 4 5 6 7 8], shape=(8,), dtype=int32)
```

Flatten the tensor

[1, 2, 3, 4]
[5, 6, 7, 8]



[1, 2, 3, 4, 5, 6, 7, 8]

```
x = tf.constant([[1,2,3,4],[5,6,7,8]], shape=(8))  
print(x)
```

```
tf.Tensor([1 2 3 4 5 6 7 8], shape=(8,), dtype=int32)
```

```
x = tf.constant([[1,2,3,4],[5,6,7,8]])  
tf.reshape(x, (8))
```

```
<tf.Tensor: shape=(8,), dtype=int32, numpy=array([1, 2, 3, 4, 5, 6, 7, 8])>
```

Reshape the tensor with (-1)

```
x = tf.constant([[1,2,3,4],[5,6,7,8]])  
print(x)
```

```
tf.Tensor(  
[[1 2 3 4]  
 [5 6 7 8]], shape=(2, 4), dtype=int32)
```

```
[ 1, 2, 3, 4]  
[ 5, 6, 7, 8]
```

```
tf.reshape(x, (-1))
```

```
[ 1, 2, 3, 4, 5, 6, 7, 8]
```

```
<tf.Tensor: shape=(8,), dtype=int32, numpy=array([1,2,3,4,5,6,7,8])>
```

Adjusts the remaining dimension to fit the data

More on (-1) shape

```
x = tf.constant([[1,2,3,4],[5,6,7,8]])  
print(x)
```

```
tf.Tensor(  
[[1 2 3 4]  
 [5 6 7 8]], shape=(2, 4), dtype=int32)
```

```
[ 1, 2, 3, 4]  
[ 5, 6, 7, 8]
```

```
tf.reshape(x, (4,-1))
```

```
<tf.Tensor: shape=(4, 2), dtype=int32, numpy=  
array([[1, 2],  
       [3, 4],  
       [5, 6],  
       [7, 8]])>
```

```
[ 1, 2]  
[ 3, 4]  
[ 5, 6]  
[ 7, 8]
```

More on (-1) shape

```
x = tf.constant([1,2,3,4,5,6,7,8])  
tf.reshape(x, (2,-1,2))
```

```
<tf.Tensor: shape=(2, 2, 2), dtype=int32, numpy=  
array([[[1, 2],  
        [3, 4]],  
  
       [[5, 6],  
        [7, 8]])>
```

```
[[ 1, 2]  
 [ 3, 4]  
 [[ 5, 6]  
 [ 7, 8]]
```

More on (-1) shape

```
x = tf.constant([1,2,3,4,5,6,7,8])  
tf.reshape(x, (2,-1,2))
```

```
<tf.Tensor: shape=(2, 2, 2), dtype=int32, numpy=  
array([[ [1, 2],  
         [3, 4]],  
       [[5, 6],  
         [7, 8]]])>
```

```
x = tf.constant([[1,2,3,4],[5,6,7,8]])  
tf.reshape(x, (-1,2,2))
```

```
<tf.Tensor: shape=(2, 2, 2), dtype=int32, numpy=  
array([[ [1, 2],  
         [3, 4]],  
       [[5, 6],  
         [7, 8]]])>
```

```
[[ 1, 2]  
 [ 3, 4]  
 [ 5, 6]  
 [ 7, 8]]
```

More on (-1) shape

```
x = tf.constant([1,2,3,4,5,6,7,8])  
tf.reshape(x, (2,-1,2))
```

```
<tf.Tensor: shape=(2, 2, 2), dtype=int32, numpy=  
array([[[1, 2],  
        [3, 4]],  
       [[5, 6],  
        [7, 8]]])>
```

```
[[ 1, 2]  
 [ 3, 4]  
 [[ 5, 6]  
 [ 7, 8]]
```

```
x = tf.constant([[1,2,3,4],[5,6,7,8]])  
tf.reshape(x, (-1,2,2))
```

```
<tf.Tensor: shape=(2, 2, 2), dtype=int32, numpy=  
array([[[1, 2],  
        [3, 4]],  
       [[5, 6],  
        [7, 8]]])>
```

```
x = tf.constant([[1,2,3,4],[5,6,7,8]])  
tf.reshape(x, (2,2,-1))
```

```
<tf.Tensor: shape=(2, 2, 2), dtype=int32, numpy=  
array([[[1, 2],  
        [3, 4]],  
       [[5, 6],  
        [7, 8]]])>
```

Slicing non-contiguous elements from a tensor

```
import tensorflow as tf

# Create a sample tensor
x = tf.constant([10, 20, 30, 40, 50, 60])

# Indices of the elements you want to slice (non-
contiguous)
indices = [0, 2, 5] # pick 10, 30, 60

# Use tf.gather for non-contiguous slicing
result = tf.gather(x, indices)

print("Original Tensor:", x.numpy())
print("Sliced Tensor:", result.numpy())
```

```
import tensorflow as tf

# 2D tensor
x = tf.constant([
    [10, 11, 12],
    [20, 21, 22],
    [30, 31, 32],
    [40, 41, 42]
])

# Select non-contiguous rows: 0th and 2nd and 3rd
row_indices = [0, 2, 3]

result = tf.gather(x, row_indices, axis=0)

print("Original Tensor:\n", x.numpy())
print("\nSelected Rows:\n", result.numpy())
```

Slicing non contiguous elements from a 3D tensor

```
import tensorflow as tf
```

```
# 3D tensor: shape (3, 4, 2)
```

```
x = tf.constant([  
    [[1, 2], [3, 4], [5, 6], [7, 8]],  
    [[9, 10], [11,12], [13,14], [15,16]],  
    [[17,18],[19,20],[21,22],[23,24]]  
])
```

```
# Pick non-contiguous slices on axis 1 (columns):  
choose columns 0 and 2  
indices = [0, 2]
```

```
result = tf.gather(x, indices, axis=1)
```

```
print("Original shape:", x.shape)  
print("Sliced shape:", result.shape)  
print(result.numpy())
```

```
import tensorflow as tf
```

```
x = tf.constant([  
    [[ 1, 2], [ 3, 4]],  
    [[ 5, 6], [ 7, 8]],  
    [[ 9, 10], [11, 12]]  
])
```

```
# Select non-contiguous 3D positions  
# (i, j, k): (row, col, depth)  
positions = [  
    [0, 1, 0], # selects 3  
    [2, 0, 1], # selects 10  
    [1, 1, 1]  # selects 8  
]
```

```
result = tf.gather_nd(x, positions)  
print(result.numpy())
```

Maximum element of a Tensor

Maximum element

```
x = tf.constant([[9,2,10,4],[5,6,7,8]])  
print(tf.reduce_max(x))
```

```
tf.Tensor(10, shape=(), dtype=int32)
```

```
[ 9, 2, 10, 4]
```

```
[ 5, 6, 7 , 8]
```

Index of the Maximum element

```
x = tf.constant([[9,2,10,4],[5,6,7,8]])  
print(tf.math.argmax(x))
```

```
tf.Tensor([0 1 0 1], shape=(4,), dtype=int64)
```

```
[ 9, 2, 10, 4]
```

```
[ 5, 6, 7 , 8]
```

```
x = tf.constant([[2, 20, 30, 3, 6], [3, 11, 16, 1, 8],  
                [14, 45, 23, 5, 27]])  
print(tf.math.argmax(x))
```

```
tf.Tensor([2 2 0 2 2], shape=(5,), dtype=int64)
```

```
[ 2, 20, 30, 3, 6 ]
```

```
[ 3, 11, 16, 1, 8 ]
```

```
[ 14, 45, 23, 5, 27]
```

Minimum element of a Tensor

Minimum element

```
x = tf.constant([[9,2,10,4],[5,6,7,8]])  
print(tf.reduce_min(x))
```

```
tf.Tensor(2, shape=(), dtype=int32)
```

```
[ 9, 2, 10, 4]
```

```
[ 5, 6, 7 , 8]
```

Index of the Minimum element

```
x = tf.constant([[9,2,10,4],[5,6,7,8]])  
print(tf.math.argmax(x))
```

```
tf.Tensor([1 0 1 0], shape=(4,), dtype=int64)
```

```
[ 9, 2, 10, 4]
```

```
[ 5, 6, 7 , 8]
```

```
x = tf.constant([[2, 20, 30, 3, 6], [3, 11, 16, 1, 8],  
                [14, 45, 23, 5, 27]])  
print(x)  
print(tf.math.argmax(x))
```

```
tf.Tensor(  
[[ 2 20 30  3  6]  
 [ 3 11 16  1  8]  
 [14 45 23  5 27]], shape=(3, 5), dtype=int32)  
tf.Tensor([0 1 1 1 0], shape=(5,), dtype=int64)
```

```
[ 2, 20, 30, 3, 6 ]
```

```
[ 3, 11, 16, 1, 8 ]
```

```
[ 14, 45, 23, 5, 27]
```

Minimum/Maximum of Two Tensors

Minimum

```
x = tf.constant([0., 0., 0., 0.])  
y = tf.constant([-5., -2., 0., 3.])  
tf.math.minimum(x, y)
```

```
<tf.Tensor: shape=(4,), dtype=float32, numpy=array([-5., -2.,  0.,  0.], dtype=float32)>
```

Maximum

```
x = tf.constant([0., 0., 0., 0.])  
y = tf.constant([-5., -2., 0., 3.])  
tf.math.maximum(x, y)
```

```
<tf.Tensor: shape=(4,), dtype=float32, numpy=array([0., 0., 0., 3.], dtype=float32)>
```

Concatenation of Two Tensors

Along 0-Axis

```
x = [[1, 2, 3], [4, 5, 6]]  
y = [[7, 8, 9], [10, 11, 12]]  
tf.concat([x, y], 0)
```

```
<tf.Tensor: shape=(4, 3), dtype=int32, numpy=  
array([[ 1,  2,  3],  
       [ 4,  5,  6],  
       [ 7,  8,  9],  
       [10, 11, 12]])>
```

Along 1-Axis

```
x = [[1, 2, 3], [4, 5, 6]]  
y = [[7, 8, 9], [10, 11, 12]]  
tf.concat([x, y], 1)
```

```
<tf.Tensor: shape=(2, 6), dtype=int32, numpy=  
array([[ 1,  2,  3,  7,  8,  9],  
       [ 4,  5,  6, 10, 11, 12]])>
```

Modifying the value of a Tensor

Not a simple operation.

Possible for Variable type. But, for such operation, we would prefer to use numpy library.

Matrix Multiplication

```
import tensorflow as tf
```

```
A1 = tf.constant([[1, 2, 3, 4]])
```

```
B1 = tf.constant([[3], [4], [5], [5]])
```

```
C1 = tf.multiply(A1, B1)
```

```
tf.print(C1)
```

```
[[3 6 9 12]
```

```
 [4 8 12 16]
```

```
 [5 10 15 20]
```

```
 [5 10 15 20]]
```

```
import tensorflow as tf
```

```
A1 = tf.constant([[1, 2, 3, 4]])
```

```
B1 = tf.constant([[3], [4], [5], [5]])
```

```
C1 = tf.matmul(A1, B1)
```

```
tf.print(C1)
```

```
[[46]]
```

Implementing Feed Forward Networks using TensorFlow

Consider the credit card defaulter example

- Given the features of the users, predict whether one is likely to be a defaulter
- A data snapshot is given below:

S. No.	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	Ethnicity	Balance	Defaultee
1	14.891	3606	283	2	34	11	Male	No	Yes	Caucasian	333	0
2	106.025	6645	483	3	82	15	Female	Yes	Yes	Asian	903	0
3	104.593	7075	514	4	71	11	Male	No	No	Asian	580	0
4	148.924	9504	681	3	36	11	Female	No	No	Asian	964	0
5	55.882	4897	357	2	68	16	Male	No	Yes	Caucasian	331	0
6	80.18	8047	569	4	77	10	Male	No	No	Caucasian	1151	1

Let us see step-by-step how to proceed in building a 2-layer feed forward model that trains on this data

Step-1: Transformation of the data

- Tensor Flow expects all inputs to be numeric, scaled and model compatible
 - Neural Networks do not process strings as inputs
 - Neural Networks perform poorly if features have different ranges
 - Data needs to be separated into train and test sets

Attribute	Type	Example	Transformation needed
Income	Numeric (continuous)	14.891	Scale
Limit	Numeric	3606	Scale
Rating	Numeric	283	Scale
Cards	Numeric (count)	2	Optional scale
Age	Numeric	34	Scale
Education	Numeric (ordinal)	11	Scale
Balance	Numeric	333	Scale
Gender	Categorical	Male	Encode
Student	Categorical	No	Encode
Married	Categorical	Yes	Encode
Ethnicity	Categorical	Caucasian	One-hot encode
Defaultee	Target (binary)	0	No transform (label)

Steps to Transform Data: Loading libraries and data

```
import pandas as pd  
import numpy as np
```

```
from sklearn.preprocessing import StandardScaler  
from sklearn.model_selection import train_test_split
```

```
import tensorflow as tf  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense
```

```
df = pd.read_csv("credit_data.csv")
```

Steps to Transform Data: Generate Encodings

Generate encodings for binary as well as multiclass categorical data

```
df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})  
df["Student"] = df["Student"].map({"Yes": 1, "No": 0})  
df["Married"] = df["Married"].map({"Yes": 1, "No": 0})
```

```
df = pd.get_dummies(df, columns=["Ethnicity"], drop_first=True)
```

- drop_first=True helps avoid the dummy variable trap
- In one-hot encoding of a categorical data, one variable can be inferred from the other variables

Like for "Ethnicity", if classes are "Caucasian", "Hispanic" and "Asian", then
"Caucasian=0" and "Hispanic=0" automatically implies "Asian=1"
"Caucasian=0" and "Hispanic=1" automatically implies "Asian=0"

So one variable is redundant

Steps to Transform Data: Feature Standardization (Scaling)

First separate the features and the target

```
X = df.drop(columns=["Defaultee"])  
y = df["Defaultee"]
```

Then standardize the features as $x' = \frac{x - \mu}{\sigma}$

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Features that are scaled:

- Income
- Limit
- Rating
- Cards
- Age
- Education
- Balance

Steps to Transform Data: Test-Train Split

Randomly split into train and test (80-20)

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.2, random_state=42  
)
```

Will use a separate validation set on the train data during model training

random_state=42 controls the randomness in splitting.

Will get the same train and test data if the same value (42) is used in different runs.

Step-2: Model Building

The model comprises of the following:

- One input layer (size is automatically inferred)
- 2 hidden layers, one of size 16 and another of size 8
 - Activation of each hidden layers is ReLU
- One output layer of size 1
 - Since it is a binary classification problem

```
model = Sequential([  
    Dense(16, activation="relu", input_shape=(X_train.shape[1],)),  
    Dense(8, activation="relu"),  
    Dense(1, activation="sigmoid")  
])
```

`X_train.shape=[# of instances, # of features]` $\leftarrow$ a List

`X_train.shape[1]` = # of features (say 16)

`input_shape=(X_train.shape[1],)` creates a tuple, without comma it is an integer value

Model Building: Compilation

- Other options of loss

```
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

- categorical_crossentropy (for 1 hot encoded labels)
- sparse_categorical_crossentropy (for integer labels)
- kullback_leibler_divergence
- Regression losses and Hinge losses are also supported

- Other options of optimizer

- SGD
- RMSprop
- Adagrad
- Adadelta

- Other options of metrics

- binary_accuracy
- categorical_accuracy
- sparse_categorical_accuracy
- top_k_categorical_accuracy

For precision and recall first import metrics as

import tensorflow as tf

from tensorflow.keras import metrics, and then in model.compile use

metrics=["accuracy", metrics.Precision(name='Precision'), metrics.Recall(name='Recall')]

Step 3: Model Training

```
model.fit(  
    X_train,  
    y_train,  
    epochs=50,  
    batch_size=16,  
    validation_split=0.1  
)
```

Step 4: Model Evaluation

Returns results on test data in the dictionary “results”

```
results = model.evaluate(X_test, y_test, return_dict=True)
print(f"Loss={results['loss']} Accuracy={results['accuracy']}, Precision={results['Precision']}")
```

Step 5: Model Inference (Single instance)

If need to be inferred for a single data instance

```
new_instance = np.array([[1,14.891, 3606, 283, 2, 34, 11, 1,1,1, 333, 1, 1]])  
new_instance_scaled = scaler.transform(new_instance)  
  
prediction = model.predict(new_instance_scaled)  
print("Probability of default:", prediction)
```

Model Inference (multiple instances in csv)

```
new_df = pd.read_csv("new_data.csv")

new_df["Gender"] = new_df["Gender"].map({"Male": 1, "Female": 0})
new_df["Student"] = new_df["Student"].map({"Yes": 1, "No": 0})
new_df["Married"] = new_df["Married"].map({"Yes": 1, "No": 0})

new_df = pd.get_dummies(new_df, columns=["Ethnicity"])

feature_columns = X.columns
new_df = new_df.reindex(columns=feature_columns, fill_value=0)

new_X_scaled = scaler.transform(new_df)

probabilities = model.predict(new_X_scaled)
predicted_labels = (probabilities >= 0.5).astype(int)

new_df["Defaultee_Probability"] = probabilities
new_df["Defaultee_Prediction"] = predicted_labels

new_df.to_csv("predictions.csv", index=False)
```

Convolution Neural Networks

Introduction

- Specialized neural network for processing data that has grid like topology
 - Time series data (one dimensional)
 - Image (two dimensional)
- Found to be reasonably suitable for certain class of problems eg. computer vision
- Instead of matrix multiplication, it uses convolution in at least one of the layers

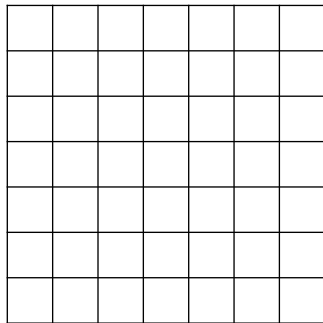
2D convolution

a	b	c	d
e	f	g	h
i	j	k	l

w	x
y	z

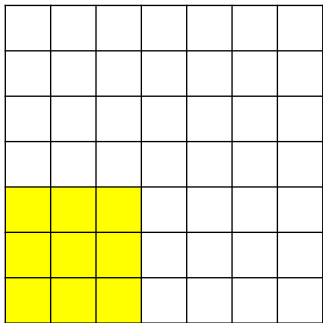
$aw+bx$ $+ey+fz$	$bw+cx$ $+fy+gz$	$cw+dx$ $+gy+hz$
$ew+fx$ $+iy+jz$	$fw+gx$ $+jy+kz$	$gw+hx$ $+ky+lz$

2D Convolution



Grid size: 7×7

2D Convolution

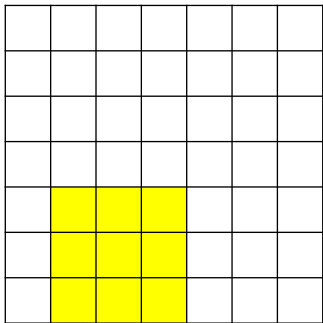


Grid size: 7×7

Filter size: 3×3

Stride: 1

2D Convolution

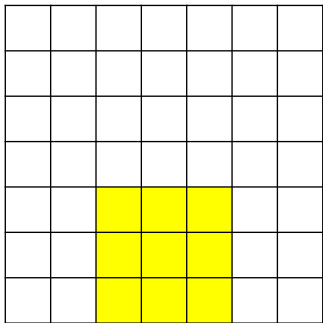


Grid size: 7×7

Filter size: 3×3

Stride: 1

2D Convolution

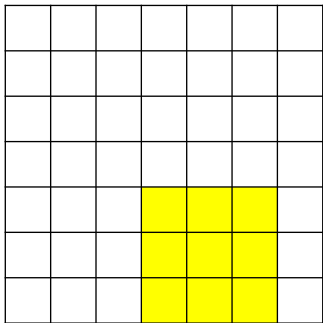


Grid size: 7×7

Filter size: 3×3

Stride: 1

2D Convolution

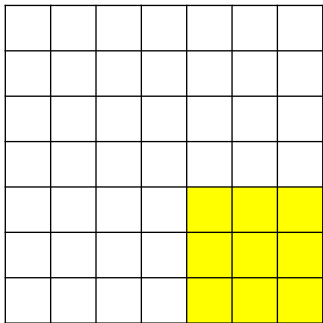


Grid size: 7×7

Filter size: 3×3

Stride: 1

2D Convolution

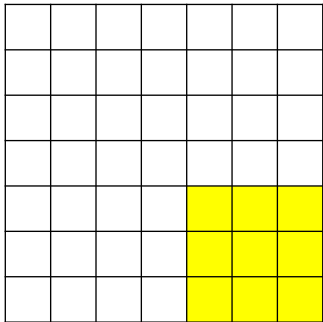


Grid size: 7×7

Filter size: 3×3

Stride: 1

2D Convolution



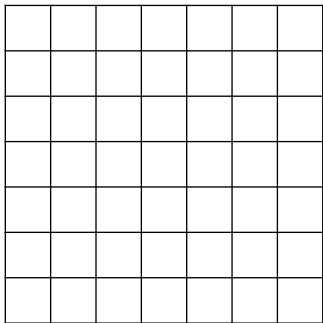
Grid size: 7×7

Filter size: 3×3

Stride: 1

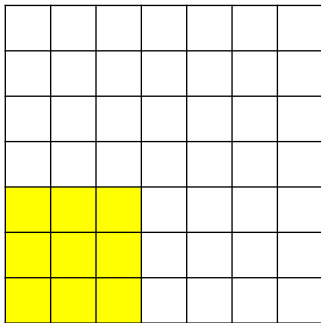
Output size: 5×5

2D convolution with stride



Grid size: 7×7

2D convolution with stride

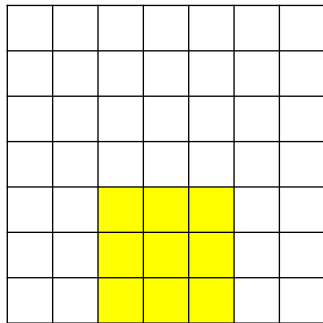


Grid size: 7×7

Filter size: 3×3

Stride: 2

2D convolution with stride

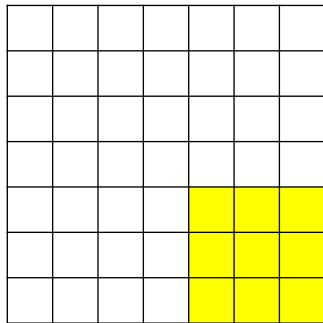


Grid size: 7×7

Filter size: 3×3

Stride: 2

2D convolution with stride

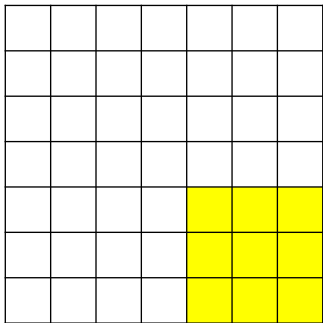


Grid size: 7×7

Filter size: 3×3

Stride: 2

2D convolution with stride



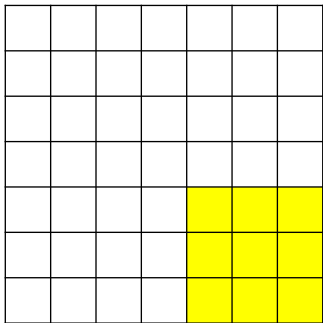
Grid size: 7×7

Filter size: 3×3

Stride: 2

Output size: 3×3

2D convolution with stride



Grid size: 7×7

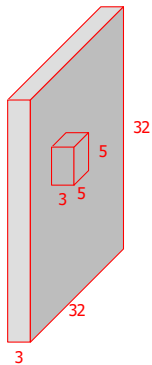
Filter size: 3×3

Stride: 2

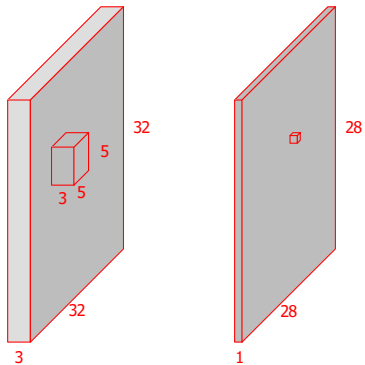
Output size: 3×3

Output size: $(N - F)/S + 1$ N -
input size, F - Filter size, S -
Stride

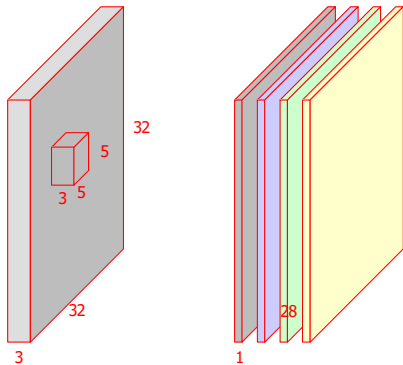
Convolution operation



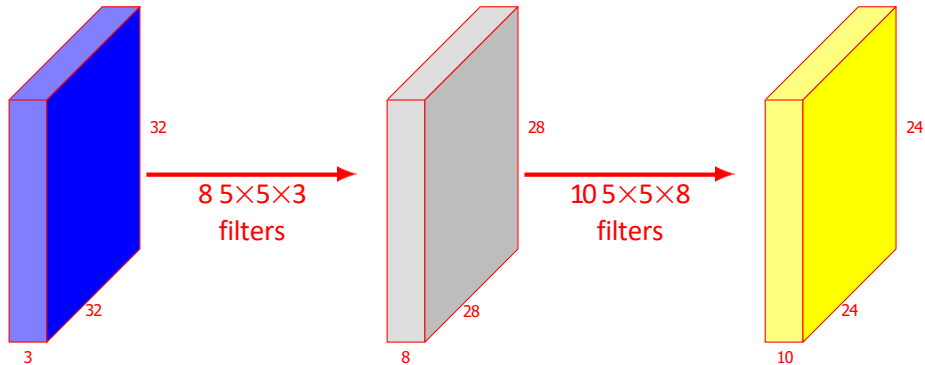
Convolution operation



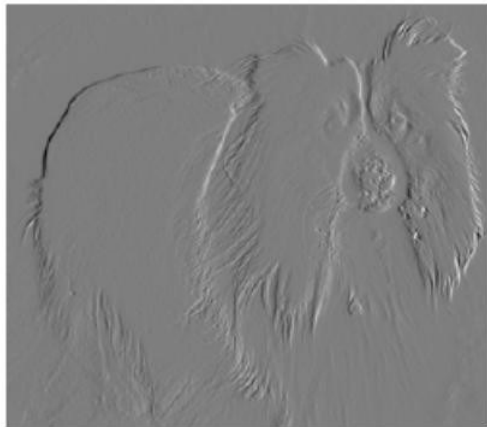
Convolution operation



Convolution example



Edge detection



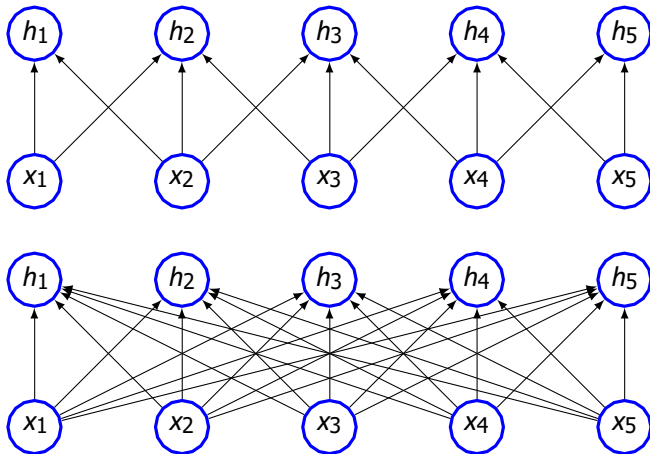
Advantages

- **Convolution can exploit the following properties**
 - Sparse interaction (Also known as sparse connectivity or sparse weights)
 - Parameter sharing
 - Equivariant representation

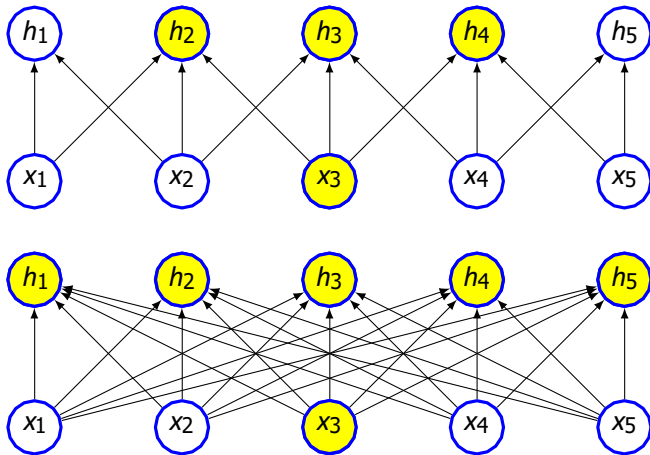
Sparse interaction

- Traditional neural network layers use matrix multiplication to describe how outputs and inputs are related
- Convolution uses a smaller kernel
 - Significant reduction in number of parameters
 - Computing output require few comparison
- For example, if there is m inputs and n outputs, traditional neural network will require $m \times n$ parameters
- If each of the output is connected to at most k units, the number of parameters will be $k \times n$

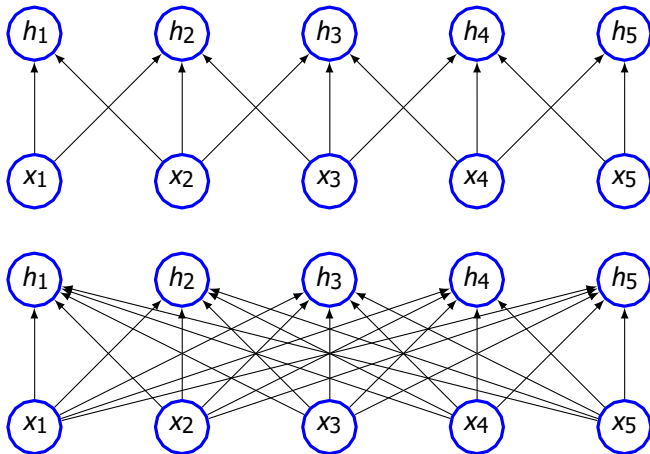
Sparse connectivity



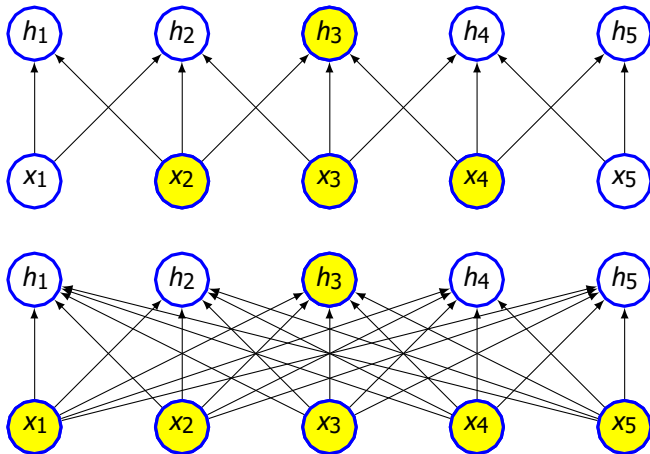
Sparse connectivity



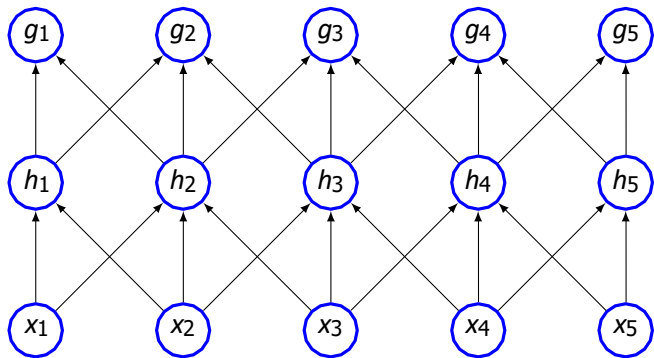
Sparse connectivity



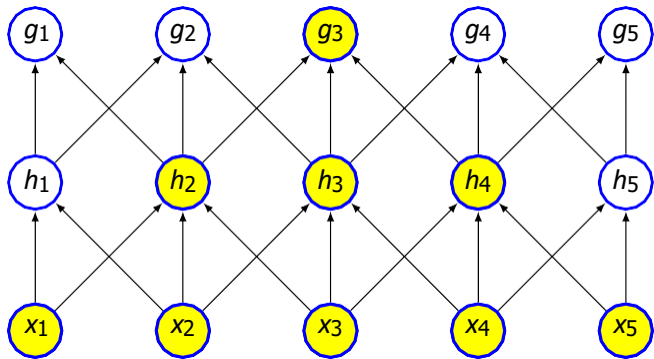
Sparse connectivity



Receptive field



Receptive field



Parameter sharing

- Same parameters are used for more than one function model
- In tradition neural network, weight is used only once
- Each member of kernel is used at every position of the inputs
- As $k \ll m$, the number of parameters will reduced significantly
- Also, require less memory

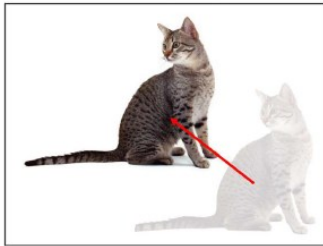
Shift Invariance

Input x



Output $f(x) = 1$

Shifted input $S_v x$

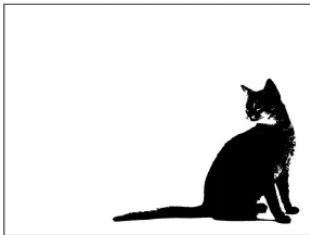


Output $f(S_v x) = 1$

- 'Cat detector' $f: \mathbb{R}^d \rightarrow \mathbb{R}$

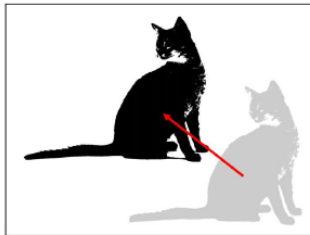
Shift Equivariance

Input x



$$\text{Output } f_i(x) = \begin{cases} 1 & \text{pixel } i \in \text{cat} \\ 0 & \text{otherwise} \end{cases}$$

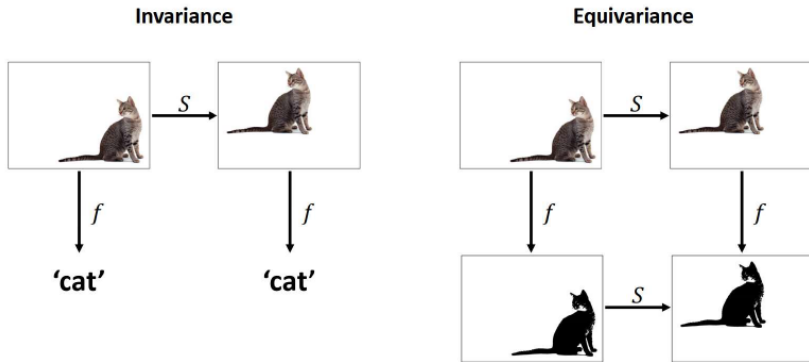
Shifted input $S_v x$



$$\text{Output } f_i(S_v x) = \begin{cases} 1 & \text{pixel } i \in \text{cat} \\ 0 & \text{otherwise} \end{cases}$$

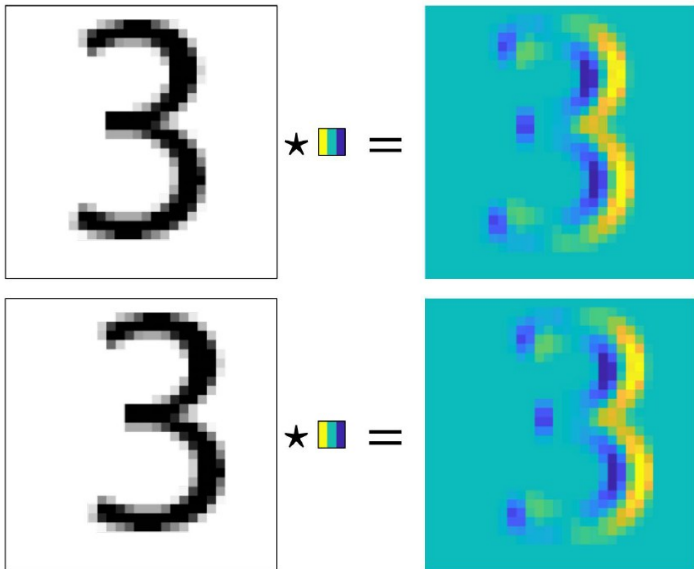
- 'Cat segmentor' $f: \mathbb{R}^d \rightarrow \mathbb{R}^d$
- Shift operator $S_v: \mathbb{R}^d \rightarrow \mathbb{R}^d$ shifting the image by vector v

Invariance vs. Equivariance

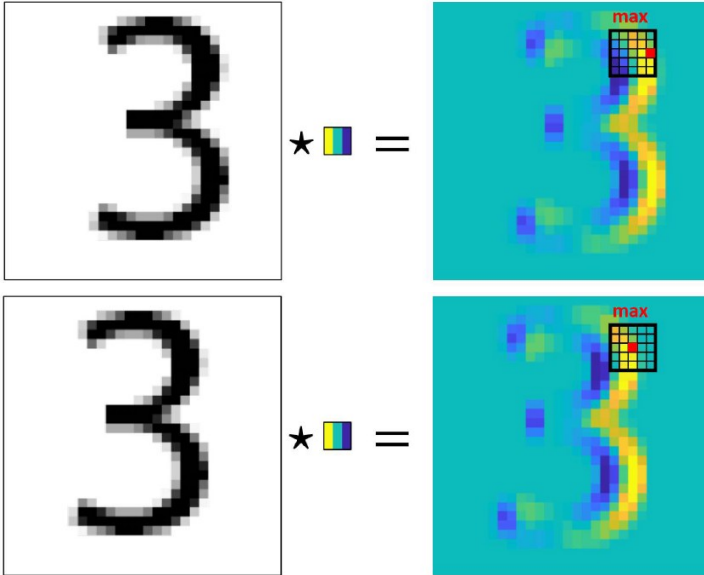


- Convolutional Networks provide Shift Equivariance
- Striding and Pooling provides mild Shift Invariance

Equivariance of CNNs -



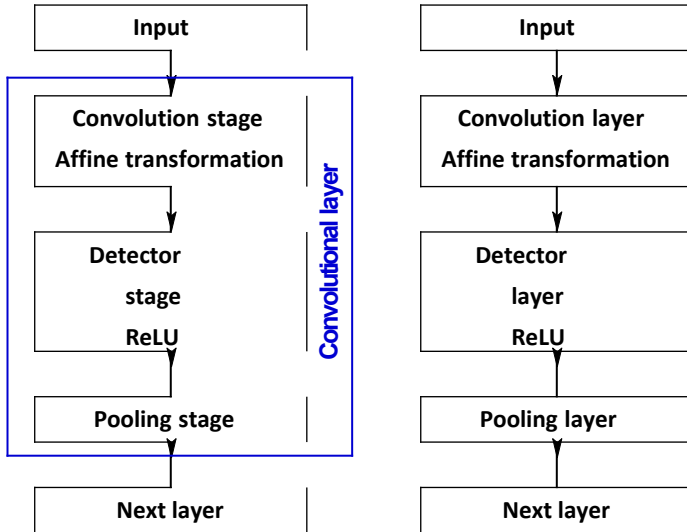
Invariance in CNNs - Pooling



Pooling

- Typical convolutional network has three stages
 - Convolution — several convolution to produce linear activation
 - Detector stage — linear activation runs through the non-linear unit such as ReLU
 - Pooling — Output is updated with a summary of statistics of nearby inputs
 - Maxpooling reports the maximum output within a rectangular neighbourhood
 - Average of rectangular neighbourhood
 - Weighted average using central pixel
- Pooling helps to make representation invariant to small translation
 - Feature is more important than where it is present
- Pooling helps in case of variable size of inputs

Typical CNN



Max Pool

0	4	7	8
9	2	4	5
6	7	3	4
8	2	1	5

Max Pool

0	4	7	8
9	2	4	5
6	7	3	4
8	2	1	5

8	

Max Pool

0	4	7	8
9	2	4	5
6	7	3	4
8	2	1	5

8	5

Max Pool

0	4	7	8
9	2	4	5
6	7	3	4
8	2	1	5

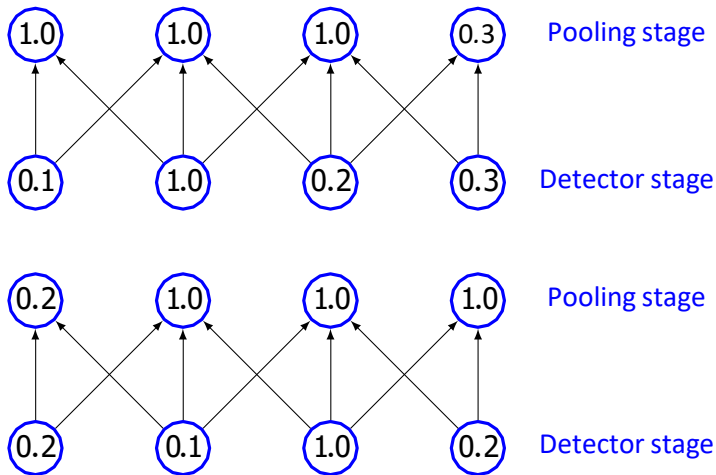
9	
8	5

Max Pool

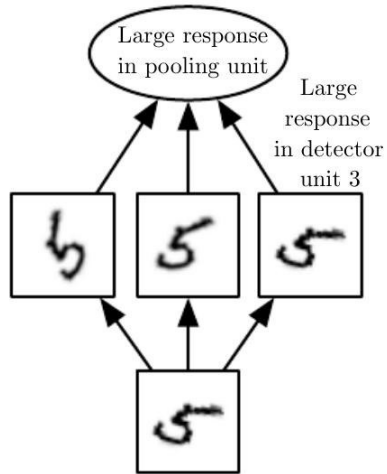
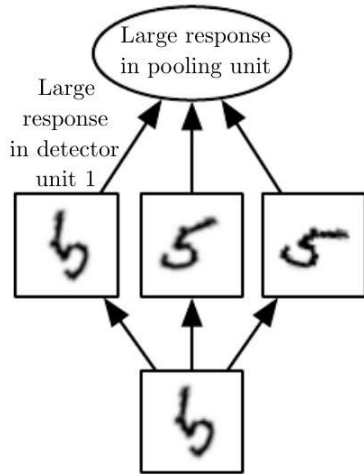
0	4	7	8
9	2	4	5
6	7	3	4
8	2	1	5

9	8
8	5

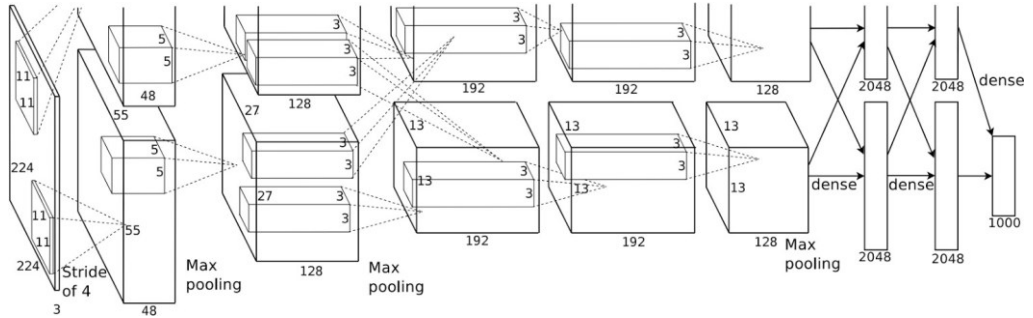
Invariance of maxpooling



Learned invariances



AlexNet



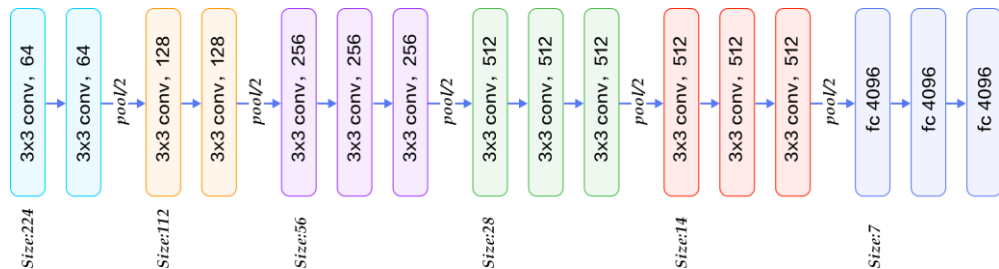
AlexNet

- Architecture

- **INPUT** - $227 \times 227 \times 3$
- **CONV1** - 96 11×11 filters at stride 4, pad 0, Output: $55 \times 55 \times 96$
- **MAX POOL1** - 3×3 filter, stride 2
Output: $27 \times 27 \times 96$
- **NORM1** - Output: $27 \times 27 \times 96$
- **CONV2** - 256 5×5 filters at stride 1, pad 2, Output: $27 \times 27 \times 256$
- **MAX POOL2** - 3×3 filter, stride 2
Output: $13 \times 13 \times 256$
- **NORM2** - $13 \times 13 \times 256$

- **CONV3** - 384 3×3 filter, stride 1, pad 1, Output: $13 \times 13 \times 384$
- **CONV4** - 384 3×3 filter, stride 1, pad 1, Output: $13 \times 13 \times 384$
- **CONV5** - 256 3×3 filter, stride 1, pad 1, Output: $13 \times 13 \times 256$
- **MAX POOL3** - 3×3 filter, stride 2, Output: $6 \times 6 \times 256$
- **FC6** - 4096 Neurons
- **FC7** - 4096 Neurons
- **FC8** - 1000 Neurons

VggNet



VGGNet improves over AlexNet by

- Using deeper architectures with uniform 3x3 convolutions
 - Enables richer hierarchical feature learning with fewer parameters per layer and
 - better transfer learning performance.

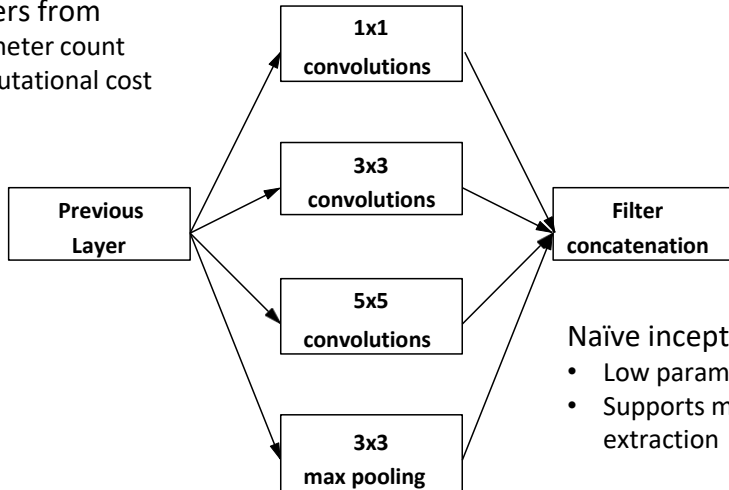
Drawbacks of AlexNet

- Very large number of parameters (~138M for VGG-16)
- High memory usage
- Slower training

Naive inception

VggNet suffers from

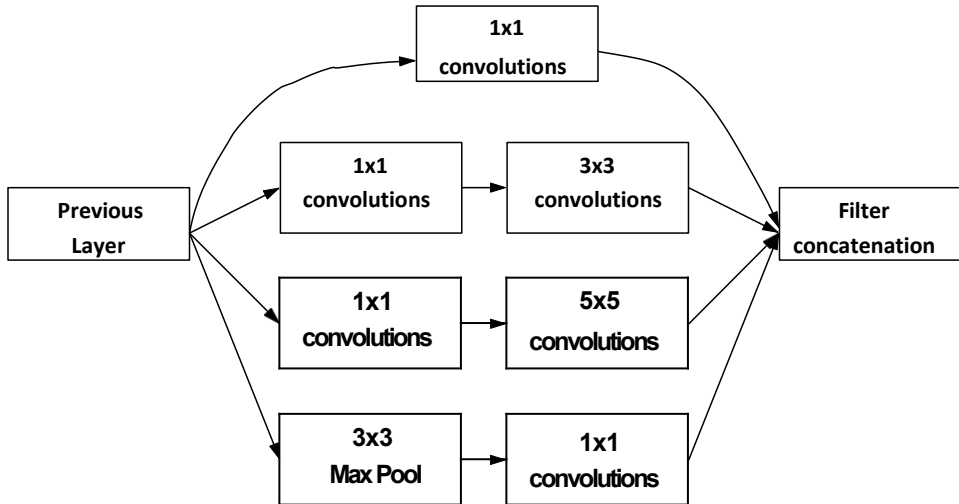
- High parameter count
- High computational cost

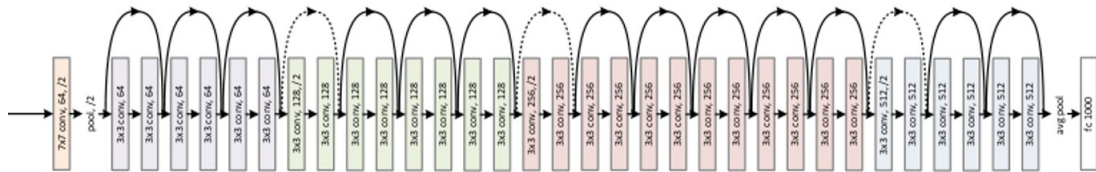


Naïve inception

- Low parameter count 5M
- Supports multiscale feature extraction

Inception





Inception

- Improves learning by width
- Uses parallel multi-scale convolutions
- Focuses on *what* features to extract

ResNet

- Improves learning by depth
- Uses skip (residual) connections
- Focuses on *how* to train very deep networks

Comparison of CNN architecture

Model	Size (M)	Top-1/top-5 error (%)	# layers	Model description
AlexNet	238	41.00/18.00	8	5 conv + 3 fc layers
VGG-16	540	28.07/9.33	16	13 conv + 3 fc layers
VGG-19	560	27.30/9.00	19	16 conv + 3 fc layers
GoogleNet	40	29.81/10.04	22	21 conv + 1 fc layers
ResNet-50	100	22.85/6.71	50	49 conv + 1 fc layers
ResNet-152	235	21.43/3.57	152	151 conv + 1 fc layers

More state of the art CNN architectures include (For self study)

- DenseNet
- Efficient-Net
- ConvNeXt

Implementing CNNs using TensorFlow

Dataset

Assume we have the Bridge Dataset

- Images of Crack and No-crack data
- Train a CNN model on the data for classification
- Make inferences on unseen data

TensorFlow pipeline

- Organize / label the dataset
- Create train–validation–test splits
- Load images
- Preprocess images (resize, normalize)
- Build a CNN model
- Compile the model
- Train with validation
- Evaluate on test set

Dataset Organization

Keras requires labels and a logical separation of the image classes

One possible organization of folders:

- datasets/
 - crack/
 - img1.jpg
 - img2.jpg
 - ...
 - no-crack/
 - img3.jpg
 - img4.jpg
 - ...

Import Libraries

```
import tensorflow as tf  
from tensorflow.keras import layers, models  
from tensorflow.keras.callbacks import EarlyStopping
```

Define parameters

```
IMAGE_SIZE = (224, 224)  
BATCH_SIZE = 32  
SEED = 42
```

- (224, 224): Common CNN size
 - If images are smaller or larger, then preprocess to standard size

Load Train and Validation data

```
train_val_ds = tf.keras.utils.image_dataset_from_directory(  
    "dataset",  
    validation_split=0.3, #Reserve 30% for validation and testing  
    subset="training",  
    seed=SEED,  
    image_size=IMAGE_SIZE,  
    batch_size=BATCH_SIZE  
)
```

At this point the code

- Reads all images
- Randomly shuffles them
- Keeps 70% for training
- Stores remaining 30% internally

The training dataset `train_val_ds` is created

Load Test and Validation Data

```
val_test_ds = tf.keras.utils.image_dataset_from_directory(  
    "dataset",  
    validation_split=0.3,  
    subset="validation",  
    seed=SEED,  
    image_size=IMAGE_SIZE,  
    batch_size=BATCH_SIZE  
)
```

This line loads ONLY the validation portion (30%) of the dataset, and stores it in a dataset called val_test_ds.

Despite the name, this dataset contains NO test data yet (unlike the training dataset, that uses a similar code)

Split validation and test dataset

```
val_batches = tf.data.experimental.cardinality(val_test_ds).numpy()  
test_ds = val_test_ds.take(val_batches // 2)  
val_ds = val_test_ds.skip(val_batches // 2)
```

- Line 1 computes number of batches in val_test_ds
- Line 2 takes first half of the batches and assigns to test_ds
- Line 3 skips the first half and takes second half of the batches and assigns to val_ds
- You may also check the class names in training data using the code below

```
class_names = train_val_ds.class_names  
print(class_names)
```

Image Normalization

```
normalization_layer = layers.Rescaling(1./255)
train_ds = train_val_ds.map(lambda x, y: (normalization_layer(x), y))
val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))
test_ds = test_ds.map(lambda x, y: (normalization_layer(x), y))
```

- Line 1 normalizes image pixel with input range [0,255] to [0,1]. Improves learning stability
- Line 2,3,4 applies image normalization to every batch of images in the training, validation and training dataset while leaving the labels unchanged.
- Each dataset like train_val_ds is a tf.data.Dataset where each element is a batch of images
 - A tensor with shape (batch_size, height, width, channels)
- lambda arguments: expression is equivalent to def function(arguments): return expression

Build CNN model

```
model = models.Sequential([  
    layers.Conv2D(32, (3,3), activation="relu", input_shape=(224,224,3)),  
    layers.MaxPooling2D(),  
  
    layers.Conv2D(64, (3,3), activation="relu"),  
    layers.MaxPooling2D(),  
  
    layers.Conv2D(128, (3,3), activation="relu"),  
    layers.MaxPooling2D(),  
  
    layers.Flatten(),  
    layers.Dense(128, activation="relu"),  
    layers.Dense(1, activation="sigmoid")  
])
```

creates a 2-D convolutional layer that learns 32 feature maps using 3×3 filters, applies a ReLU activation, and expects 224×224 RGB images as input.

Compiling the model

```
model.compile(  
    optimizer="adam",  
    loss="binary_crossentropy",  
    metrics=["accuracy"]  
)
```

Early Stopping Setup and Training

```
early_stop = EarlyStopping(  
    monitor="val_loss",  
    patience=5,  
    restore_best_weights=True  
)
```

```
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=30,  
    callbacks=[early_stop]  
)
```

Evaluate on Test Data

```
test_loss, test_accuracy = model.evaluate(test_ds)  
print("Test accuracy:", test_accuracy)
```

Make predictions on new data

```
import numpy as np
from tensorflow.keras.preprocessing import image

img = image.load_img("new_bridge.jpg", target_size=IMAGE_SIZE)
img_array = image.img_to_array(img) #Converts image to numPy array size 224x224x3
img_array = np.expand_dims(img_array, axis=0) #CNN expects batch size so expands to
<batchsize, height, width, num_of_channels> i.e., (1,224,224,3)
img_array = img_array / 255.0

prediction = model.predict(img_array)

if prediction[0][0] > 0.5:
    print("No Crack")
else:
    print("Crack")
```

How to use pretrained models like VggNet & ResNet

Import Libraries

```
import tensorflow as tf  
from tensorflow.keras.applications import VGG16, ResNet50  
from tensorflow.keras.applications.vgg16 import preprocess_input  
from tensorflow.keras import layers, models
```

Dataset Parameters

```
IMAGE_SIZE = (224, 224)
```

```
BATCH_SIZE = 32
```

```
SEED = 42
```

Load Training Dataset

```
train_ds = tf.keras.utils.image_dataset_from_directory(  
    'dataset',  
    validation_split=0.3,  
    subset='training',  
    seed=SEED,  
    image_size=IMAGE_SIZE,  
    batch_size=BATCH_SIZE  
)
```

Load Validation & Test Dataset

```
temp_ds = tf.keras.utils.image_dataset_from_directory(  
    'dataset',  
    validation_split=0.3,  
    subset='validation',  
    seed=SEED,  
    image_size=IMAGE_SIZE,  
    batch_size=BATCH_SIZE  
)  
  
val_size = tf.data.experimental.cardinality(temp_ds) // 2  
val_ds = temp_ds.take(val_size)  
test_ds = temp_ds.skip(val_size)
```

Load Pretrained VGG16

```
base_model = VGG16(  
    weights='imagenet',  
    include_top=False,  
    input_shape=(224, 224, 3)  
)  
  
base_model.trainable = False
```

Build Classifier Head

```
model = models.Sequential([  
    base_model,  
    layers.GlobalAveragePooling2D(),  
    layers.Dense(128, activation='relu'),  
    layers.Dropout(0.5),  
    layers.Dense(1, activation='sigmoid')  
])
```

Compile Model

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

Apply Preprocessing

```
train_ds = train_ds.map(lambda x, y: (preprocess_input(x), y))  
val_ds = val_ds.map(lambda x, y: (preprocess_input(x), y))  
test_ds = test_ds.map(lambda x, y: (preprocess_input(x), y))
```

Train the Model

```
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=10  
)
```

Evaluate on Test Set

```
test_loss, test_acc = model.evaluate(test_ds)  
print('Test Accuracy:', test_acc)
```

Prediction on New Images

```
predict_ds = tf.keras.utils.image_dataset_from_directory(  
    'predict_images',  
    labels=None,  
    shuffle=False,  
    image_size=IMAGE_SIZE,  
    batch_size=BATCH_SIZE  
)  
  
predictions = model.predict(predict_ds)
```